

人工智能技术

Artificial Intelligence

——人工智能：经典智能+智能计算+机器学习

AI: Classical Intelligence + Computing Intelligence + Machine Learning

王鸿鹏、王润花、韩明静、
许丽、靖智博
南开大学人工智能学院





强化学习

REINFORCEMENT LEARNING

智能体根据奖励与惩罚的经验学习，以使未来的奖励最大化

从奖励中学习

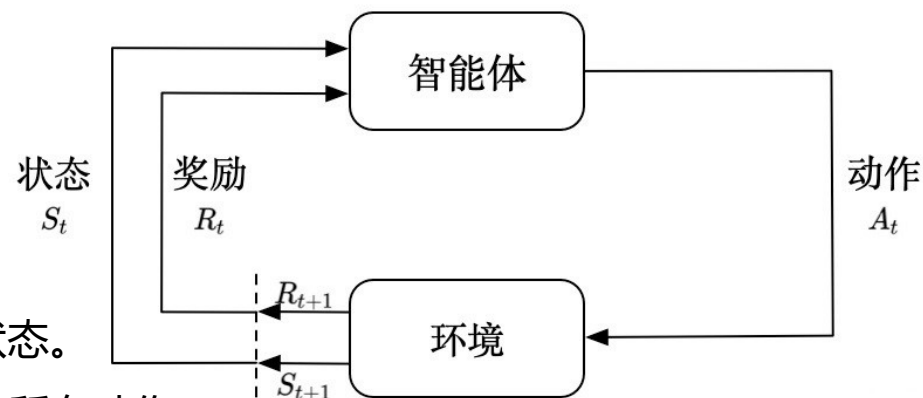
强化学习：智能体与世界进行互动并定期收到反映其表现的奖励（即强化）的学习方法

强化学习基本结构

- **智能体**：强化学习的本体，作为学习者或者决策者。
- **环境**：强化学习智能体以外的一切，主要由状态集合组成。
- **状态**：一个表示环境的数据，状态集则是环境中所有可能的状态。
- **动作**：智能体可以做出的动作，动作集则是智能体可以做出的所有动作。
- **奖励**：智能体在执行一个动作后，获得的正/负反馈信号，奖励集则是智能体可以获得的所有反馈信息。
- **策略**：强化学习是从环境状态到动作的映射学习，称该映射关系为策略。通俗的理解，即智能体如何选择动作的思考过程称为策略。
- **目标**：智能体自动寻找在连续时间序列里的最优策略，而最优策略通常指最大化长期累积奖励。

智能体3种关键要素：感知、决策和奖励

因此，强化学习实际上是智能体在与环境进行交互的过程中，学会最佳决策序列。



从奖励中学习

强化学习基本概念

- 随机状态转移:

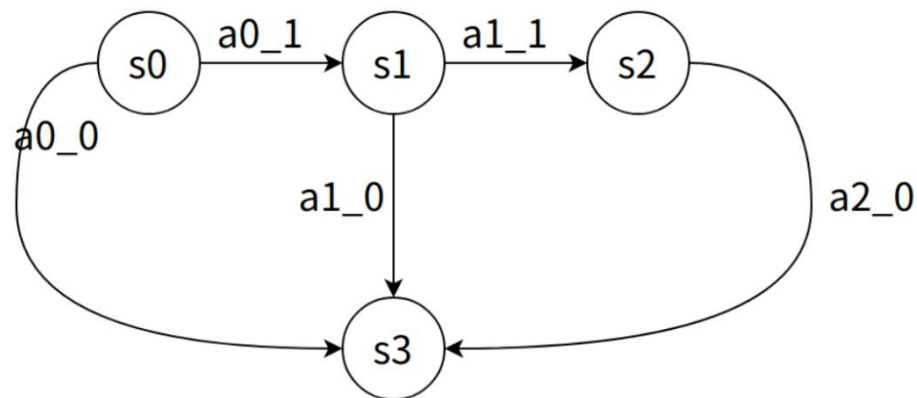
$$p_t(s' | s, a) = P(S'_{t+1} = s' | S_t = s, A_t = a)$$

表示发生下述事件的概率：在当前状态 s ，智能体执行动作 a ，环境的状态变成 s' 。

- 确定状态转移:

$$p_t(s' | s, a) = \begin{cases} 1, & \text{if } \tau_t(s, a) = s' \\ 0, & \text{otherwise.} \end{cases}$$

表示发生下述事件：在当前状态 s ，智能体执行动作 a ，环境的状态变成 s' 。



从奖励中学习

强化学习基本概念

- 随机策略:

$$\begin{cases} 0 \leq \pi(a|s) < 1, & \forall a \in A \\ \sum_{a \in A} \pi(a|s) = 1 \end{cases}$$

- 确定策略:

$$\pi(a|s) = \begin{cases} 1, & \text{if } \mu(s) = a; \\ 0, & \text{otherwise.} \end{cases}$$

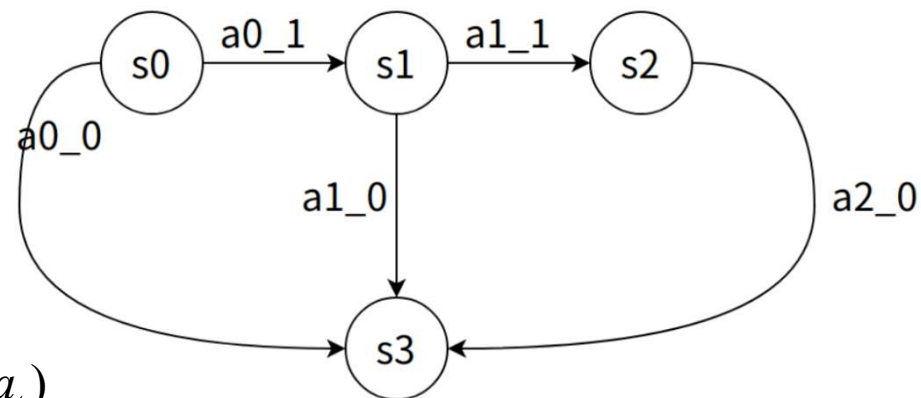
ϵ -greedy策略

$$\pi^{\hat{a}} = \text{*}argmax_{\pi} Q_{\pi}(s_t, a_t)$$

$$\pi(a|s) \leftarrow \begin{cases} 1 - \epsilon + \epsilon / |A(s)| & \text{if } a = \pi^{\hat{a}} \\ \epsilon / |A(s)| & \text{if } a \neq \pi^{\hat{a}} \end{cases}$$

greedy策略

$$\pi(a|s) \leftarrow \begin{cases} 1 & \text{if } a = \pi^{\hat{a}} \\ 0 & \text{if } a \neq \pi^{\hat{a}} \end{cases}$$



从奖励中学习

强化学习基本概念

- 奖励:

在t时刻状态执行动作以后，环境所能给予的即时反馈

确定的奖励

$$r_t = r(s_t, a_t)$$

不确定的奖励(随机变量)

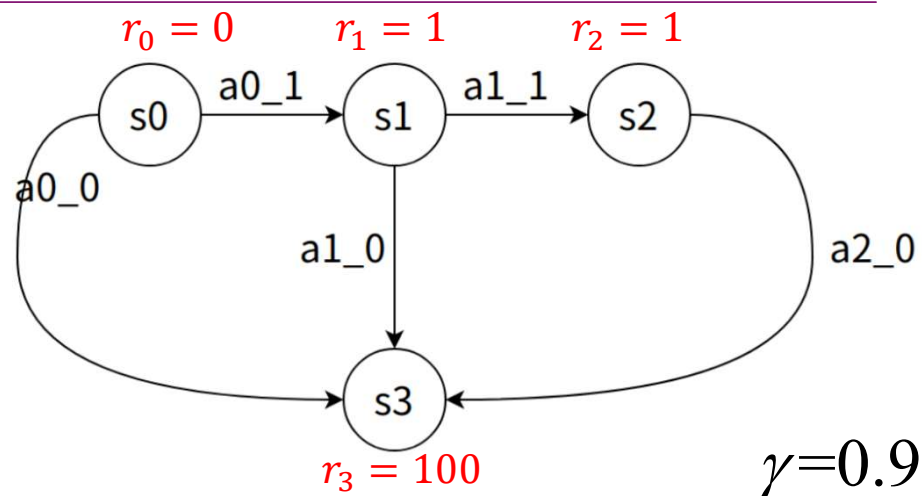
$$R_t = r(s_t, A_t) \quad \text{或} \quad R_t = r(S_t, A_t)$$

- 回报函数:

t时刻执行动作直至达到终止条件，获得的所有奖励总和。

折扣回报

$$U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_n + \cdots. \quad U_t = R_t + \gamma^1 R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \cdots + \gamma^n R_n + \cdots.$$



从奖励中学习

强化学习基本概念

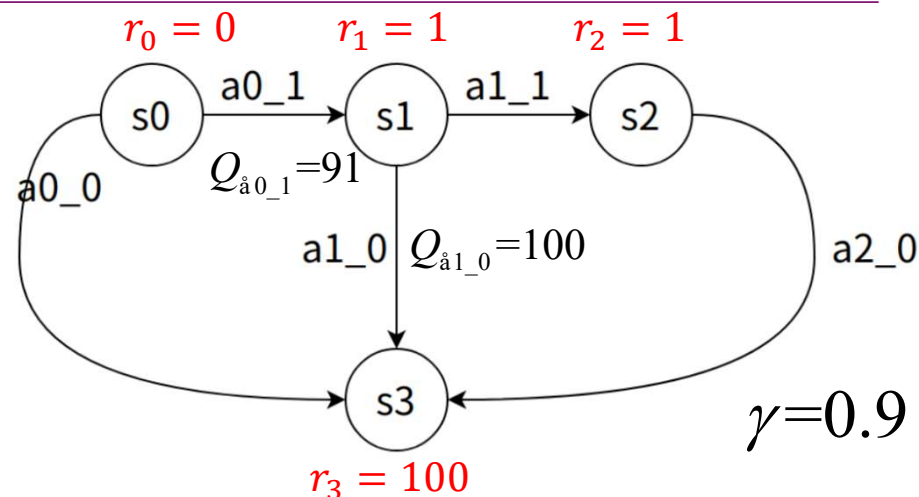
- 动作价值:

状态s执行动作a对后续获得的回报变量求期望

$$Q_{\pi}(s_t, a_t) = E_{S_{t+1}, A_{t+1}, \dots, S_n, A_n} [U_t | S_t = s_t, A_t = a_t].$$

- 状态价值:

$$V_{\pi}(s_t) = E_{A_t, S_{t+1}, A_{t+1}, \dots, S_n, A_n} [U_t | S_t = s_t]$$



最优动作价值函数

$$Q_a(s_t, a_t) = \max_{\pi} Q_{\pi}(s_t, a_t), \quad \forall s_t \in S, \quad a_t \in A.$$

$$\pi^a = \text{*argmax}_{\pi} Q_{\pi}(s_t, a_t), \quad \forall s_t \in S, \quad a_t \in A.$$

$$\begin{aligned} V_{\pi}(s_t) &= E_{A_t \sim \pi(\cdot | s_t)} [Q_{\pi}(s_t, A_t)] \\ &= \sum_{a \in A} \pi(a | s_t) \cdot Q_{\pi}(s_t, a). \end{aligned}$$

从奖励中学习

方法分类:

- **基于模型的强化学习:** 智能体使用环境的**转移模型**来帮助解释奖励信号并决定如何行动
 - 模型最初可能是**未知**的: 智能体通过观测其行为的影响来学习模型
 - 在**部分可观测**的环境中, 转移模型对于状态估计也是很用的
 - **效用函数 $U(s)$** : 状态 s 之后的奖励总和
- **无模型强化学习:** 智能体不知道环境的转移模型, 并且也不会学习它, 直接学习**如何采取行为方式**
 - 动作效用函数学习: 最常见形式是Q学习(Q-learning), 即智能体学习Q函数 (Q-function, 或称质量函数) $Q(s, a)$, 这个函数代表从状态 s 出发, 采取动作 a 的奖励的总和。给定一个Q函数, 智能体可以通过寻找具有最高Q值的动作来决定在状态 s 下采取的行动。
 - 策略搜索: 智能体将学习一个策略 $\pi(s)$ 即从状态到动作的直接映射。是一个反射型智能体。

被动强化学习

被动强化学习

被动学习智能体: 智能体学习效用函数 $U^\pi(s)$, 从状态 s 出发, 采用策略 π 所得到的期望总折扣奖励。

- 具有少量动作和状态, 且环境完全可观测, 其中智能体已经有了能决定其动作的固定策略 $\pi(s)$
- 被动学习智能体并不知道转移模型 $P(s' | s, a)$, 即在状态 s 下采取动作 a 后到达状态 s' 的概率
- 不知道奖励函数 $R(s, a, s')$, 即每次转移后的奖励。
- 智能体在环境中根据其策略 π 执行一组**试验**。在每次试验中, 智能体从状态 $(1, 1)$ 出发, 经历一系列状态转移, 直到达到终止状态, 即 $(4, 2)$ 或 $(4, 3)$ 之一。它既能感知到当前的状态, 也能感知到达到该状态的转移所获得的奖励。典型的试验可能类似如下过程:

$$\begin{aligned}
 &(1, 1) \xrightarrow[\text{Up}]{-.04} (1, 2) \xrightarrow[\text{Up}]{-.04} (1, 3) \xrightarrow[\text{Right}]{-.04} (1, 2) \xrightarrow[\text{Up}]{-.04} (1, 3) \xrightarrow[\text{Right}]{-.04} (2, 3) \xrightarrow[\text{Right}]{-.04} (3, 3) \xrightarrow[\text{Right}]{+1} (4, 3) \\
 &(1, 1) \xrightarrow[\text{Up}]{-.04} (1, 2) \xrightarrow[\text{Up}]{-.04} (1, 3) \xrightarrow[\text{Right}]{-.04} (2, 3) \xrightarrow[\text{Right}]{-.04} (3, 3) \xrightarrow[\text{Right}]{-.04} (3, 2) \xrightarrow[\text{Up}]{-.04} (3, 3) \xrightarrow[\text{Right}]{+1} (4, 3) \\
 &(1, 1) \xrightarrow[\text{Up}]{-.04} (1, 2) \xrightarrow[\text{Up}]{-.04} (1, 3) \xrightarrow[\text{Right}]{-.04} (2, 3) \xrightarrow[\text{Right}]{-.04} (3, 3) \xrightarrow[\text{Right}]{-.04} (3, 2) \xrightarrow[\text{Up}]{-1} (4, 2)
 \end{aligned}$$

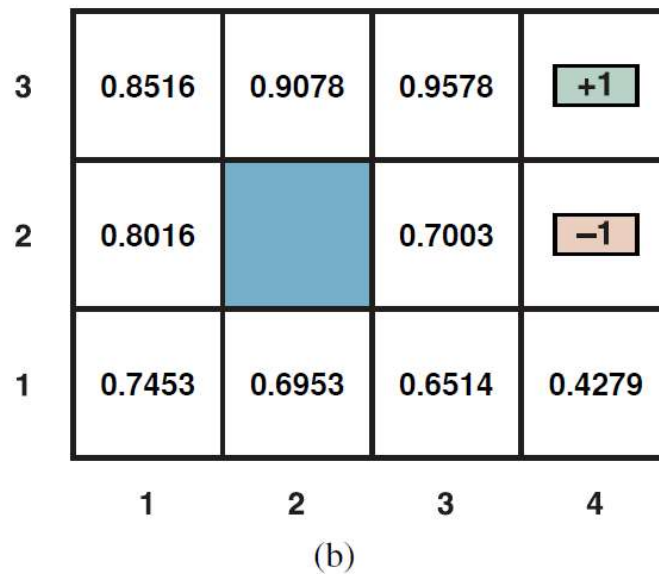
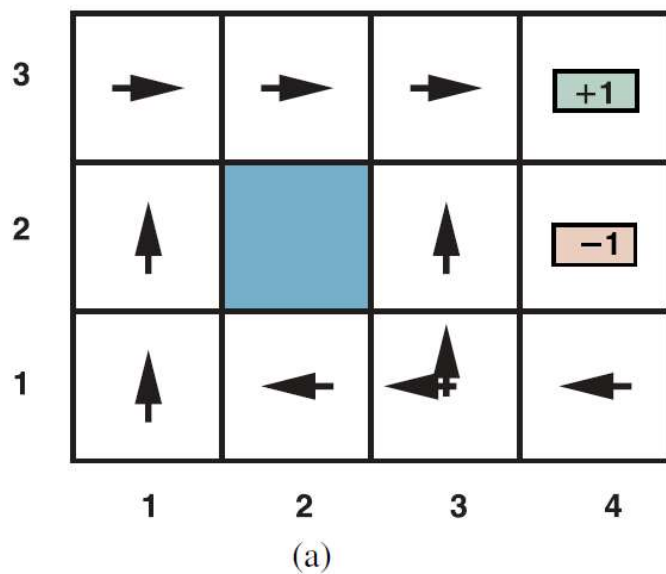
- 期望效用: 遵循策略 π 所能获得的 (折扣) 奖励的期望总和

$$U^\pi(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1}) \right], \quad R(S_t, \pi(S_t), S_{t+1}) \text{ 为在状态 } S_t \text{ 下采取动作 } \pi(S_t) \text{ 到达 } S_{t+1} \text{ 所获得的奖励}$$

被动强化学习



被动强化学习



- (a) $R(s, a, s^t)$ 的随机环境中非终止状态间转移的最优策略。其中状态(3, 1)有两种策略，因为在该状态中Left和Up都是最优的
- (b) 给定策略 π 后， 4×3 世界中各个状态的效用函数

被动强化学习

直接效用估计

- 一个状态的效用定义为从该状态出发的期望总奖励——称其为期望预期奖励 (reward-to-go) ,并且每次试验将为每个访问过的状态提供一个它的数值样本
- 在每个序列的末尾, 算法将计算每个状态的预期奖励, 并相应地, 采用平均的方法来更新每个状态的效用估计值。若试验的次数没有限制, 样本平均值将收敛到真实期望值。

(例: 第一次试验为状态(1, 1)提供了总奖励为0.76的样本, 为(1, 2)提供了总奖励为0.80和0.88的两个样本, 为状态(1, 3)提供了0.84和0.92两个样本,)

$$\begin{aligned}
 &(1, 1) \xrightarrow[\text{Up}]{-.04} (1, 2) \xrightarrow[\text{Up}]{-.04} (1, 3) \xrightarrow[\text{Right}]{-.04} (1, 2) \xrightarrow[\text{Up}]{-.04} (1, 3) \xrightarrow[\text{Right}]{-.04} (2, 3) \xrightarrow[\text{Right}]{-.04} (3, 3) \xrightarrow[\text{Right}]{+1} (4, 3) \\
 &(1, 1) \xrightarrow[\text{Up}]{-.04} (1, 2) \xrightarrow[\text{Up}]{-.04} (1, 3) \xrightarrow[\text{Right}]{-.04} (2, 3) \xrightarrow[\text{Right}]{-.04} (3, 3) \xrightarrow[\text{Right}]{-.04} (3, 2) \xrightarrow[\text{Up}]{-.04} (3, 3) \xrightarrow[\text{Right}]{+1} (4, 3) \\
 &(1, 1) \xrightarrow[\text{Up}]{-.04} (1, 2) \xrightarrow[\text{Up}]{-.04} (1, 3) \xrightarrow[\text{Right}]{-.04} (2, 3) \xrightarrow[\text{Right}]{-.04} (3, 3) \xrightarrow[\text{Right}]{-.04} (3, 2) \xrightarrow[\text{Up}]{-1} (4, 2)
 \end{aligned}$$

- 这意味着已经将强化学习简化为一个标准的监督学习问题, 其中每个样例都是一对(状态, 预期奖励)
- **但它忽略了一个重要的约束: 状态的效用取决于后继状态的奖励和期望效用。更具体地说, 效用值应当满足固定策略的贝尔曼方程**

$$U_i(s) = \sum_{s'} P(s' | s, \pi_i(s)) [R(s, \pi_i(s), s') + \gamma U_i(s')].$$

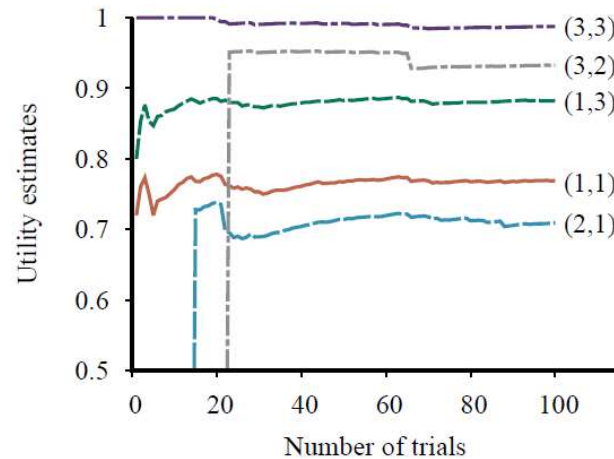
被动强化学习

自适应动态规划 (ADP)

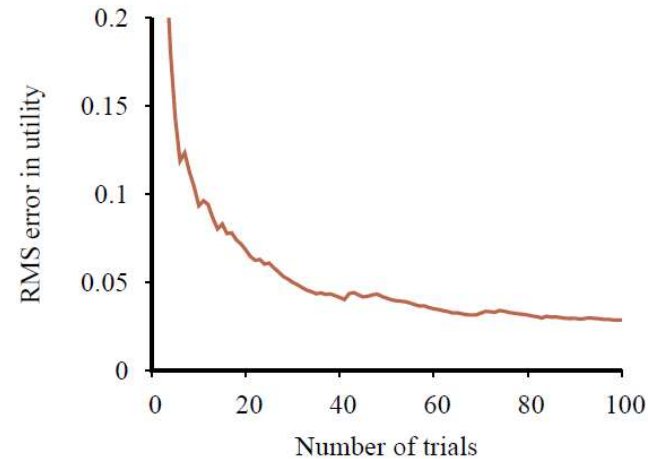
- 智能体学习状态之间的转移模型并使用动态规划解决相应的马尔可夫决策过程，从而利用了状态效用之间的约束
- 意味着可以将学习到的转移模型 $P(s' | s, \pi(s))$ 和观测到的奖励 $R(s, \pi(s), s')$ 来求解各个状态的效用。当策略 π 固定时，这些贝尔曼方程是线性的方程组，因此可以使用任意的线性代数软件包来求解

```
function PASSIVE-ADP-LEARNER(percept) returns an action
  inputs: percept, a percept indicating the current state  $s'$  and reward signal  $r$ 
  persistent:  $\pi$ , a fixed policy
                $mdp$ , an MDP with model  $P$ , rewards  $R$ , actions  $A$ , discount  $\gamma$ 
                $U$ , a table of utilities for states, initially empty
                $N_{s'|s,a}$ , a table of outcome count vectors indexed by state and action, initially zero
                $s, a$ , the previous state and action, initially null

  if  $s'$  is new then  $U[s'] \leftarrow 0$ 
  if  $s$  is not null then
    increment  $N_{s'|s,a}[s, a][s']$ 
     $R[s, a, s'] \leftarrow r$ 
    add  $a$  to  $A[s]$ 
     $\mathbf{P}(\cdot | s, a) \leftarrow \text{NORMALIZE}(N_{s'|s,a}[s, a])$ 
     $U \leftarrow \text{POLICYEVALUATION}(\pi, U, mdp)$ 
     $s, a \leftarrow s', \pi[s']$ 
  return  $a$ 
```



(a)



(b)

关于 4×3 世界问题的被动自适应动态规划的学习曲线

(a) 选定某个状态子集，其效用估计值与试验次数的关系。注意，对于很少被访问的状态(2, 1)和(3, 2)，它们分别在第14次和第23次试验才被“发现”连接到位于(4, 3)处的+1退出状态。

(b) $U(1, 1)$ 估计的均方根误差，所示的结果为50组，每组执行100次试验的平均值

被动强化学习

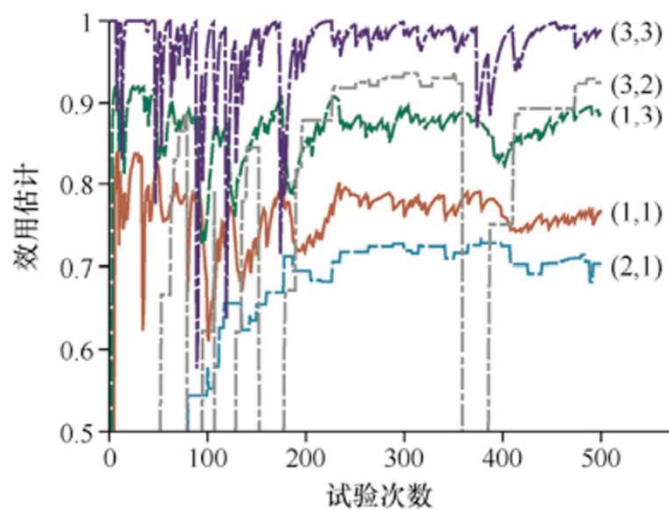
时序差分学习

- 使用观测到的转移来更改观测状态的效用，从而迫使它们满足约束方程

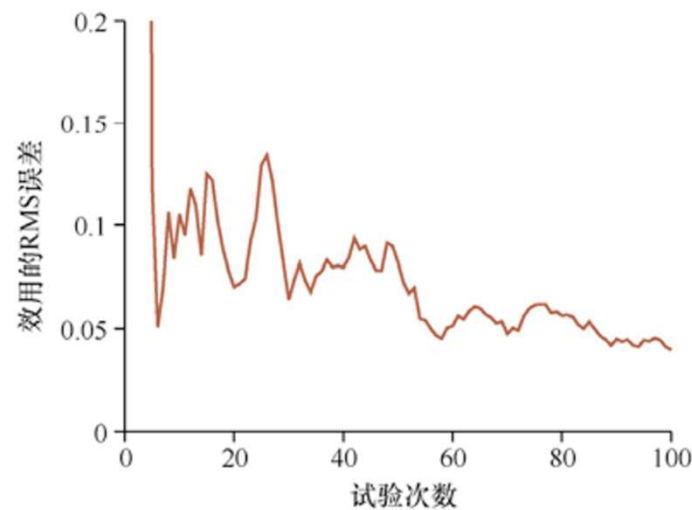
时序差分 (TD) 方程

$$U^\pi(s) \leftarrow U^\pi(s) + \alpha [R(s, \pi(s), s') + \gamma U^\pi(s') - U^\pi(s)].$$

- α 学习率参数
- 由于此更新规则使用了相继状态（以及相继时间）之间效用的差分，因此通常被称为时序差分 (temporal_difference, TD) 方程
- TD项 $R(s, \pi(s), s') + \gamma U^\pi(s') - U^\pi(s)$ 是关于误差的信息，更新旨在减少该误差
- 所有时序差分方法都通过调整理想均衡的效用估计来实现学习目的，当效用估计正确时，理想均衡是局部成立的。
- TD不需要通过转移模型来进行所需的更新。环境本身只提供由观测到的转移所给出的相邻状态之间的连接关系



(a)



(b)

4×3世界问题中，TD的学习曲线。

(a) 选定的状态子集的效用估计与试验次数的关系。此为单组执行500次试验的结果。可以与图22-3a中的每组执行100次试验的结果进行比较。

(b) U(1, 1)估计的均方根误差，所示结果为50组、每组执行100次试验的平均值

被动强化学习



TD学习与ADP学习比较

两者都试图对效用估计进行局部调整，以使每个状态与其后继状态能“达成一致”			
TD方法调整状态的效用来达到与其观测到的后继状态一致	自适应动态规划调整状态的效用来达到与所有可能发生的后继状态一致，这些状态将按概率进行加权求和	当TD的调整在大量转移上取平均时，这种差异就会消失，因为每个后继状态在转移的集合中发生的频率与其发生的概率大致成正比。	
TD对每个观测到的转移都只进行单次调整	自适应动态规划进行尽可能多的调整，以保持效用估计U和转移模型P之间的一致性	观测到的转移仅对P产生局部的改变，但它可能会影响到整个U。因此，TD可以看作对自适应动态规划的粗略但有效的近似	
对于每个观测到的转移，TD智能体将生成大量的虚构的转移。这样一来，由此产生的效用估计将越来越接近ADP的效用估计值	ADP所做的每一次调整都可以看作当前转移模型进行一次模拟产生的伪经验		

主动强化学习

主动学习智能体

被动学习智能体有一个固定的策略来决定其行为，而主动学习智能体（active learning agent）可以自主决定采取什么动作

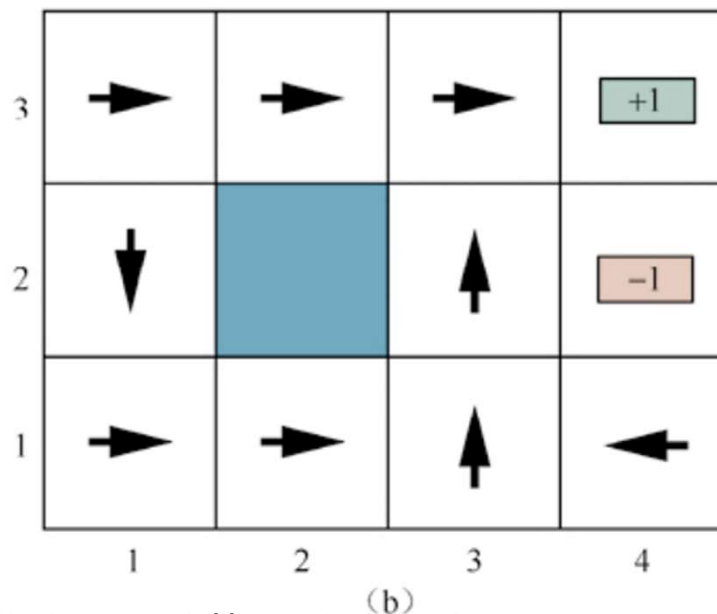
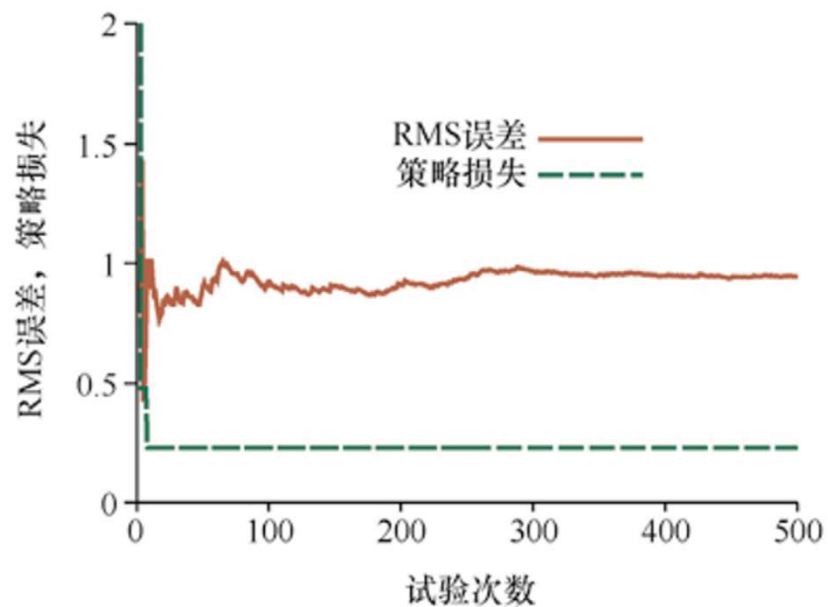
以自适应动态规划（ADP）智能体开始入手考虑如何对它进行修改以利用这种新的自由度，考虑3个问题：

- 智能体需要学习一个完整的转移模型，其中包含所有动作可能导致的结果及概率，而不仅仅是固定策略下的模型
- 智能体有一系列动作可供选择。它需要学习的效用是由最优策略所定义的效用，它们将满足贝尔曼方程

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')]$$

- 智能体在每一步中该做什么。在获得对所学到的模型最优的效用函数U后，智能体可以通过一步前瞻寻找并采取最大化期望效用的最优动作；或者，如果智能体使用的是策略迭代，那么现在已经得到了最优策略，因此它可以简单地执行最优策略建议的动作。但这样操作就正确吗？

主动强化学习



贪心ADP智能体的表现，它采取所学习到的模型下的最优策略所推荐的动作。

(a) 在所有9个非终止方格中取平均值的均方根 (RMS) 误差，以及(1, 1)中的策略损失。我们可以看到，该策略很快地，在仅仅8次试验之后，就收敛到一个损失为0.235的次优策略。

(b) 贪心智能体在这个特定的试验序列中收敛到的次优策略。注意，(1, 2)状态中采取Down动作

主动强化学习

探索

- 贪心智能体 (greedy agent) : 它每一步都贪婪地执行当前认为的最优动作 $a^* = \arg \max_a Q(s, a)$
- 贪心方法有时是有效的, 使得对应的智能体会收敛到最优策略, 但在很多情况下, 它的效果并不理想。这是由于学习到的模型与真实的环境不一样, 学习到的模型的最优结果可能在真实环境中是次优的。
- 贪心智能体忽视了: 动作不仅提供奖励, 还以结果状态中感知的形式提供信息。智能体必须在利用当前最佳动作以最大化其短期奖励和探索过去未知的状态以获得可能导致策略改变 (以及在未来获得更大奖励) 的信息之间进行权衡 (老虎机问题) 。
- 最优策略的方案对于下一步行动的选取都不应该是贪心的, 而应该是所谓 “无限探索极限下的贪心 ” (GLIE)
 - GLIE方案必须在每个状态下对每个动作尝试任意多次, 以避免错过最优动作
 - 使用这种方案的ADP智能体最终将学习到正确的转移模型, 在此基础上, 它可以在不考虑探索的情况下行动
 - 最简单的一种方案是让智能体在时刻t以1/t的概率随机选择一个动作, 否则就遵循贪心策略。一个更好的方法是对智能体不经常尝试的动作赋予较高的权重, 同时倾向于避免采取智能体认为效用较低的动作

$$U^+(s) \leftarrow \max_a f\left(\sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U^+(s')], N(s, a)\right) \quad f \text{表示探索函数}$$

主动强化学习

安全搜索

- 许多动作是**不可逆**的
 - 不存在后续的动作序列可以将状态恢复到不可逆动作发生之前的状态
 - 最坏的情况下，智能体将进入到一个吸收态（absorbing state），此时任何动作都不会改变当前的状态，智能体也不会得到任何奖励
- 一个更好的方法是选择一个对所有有机会成为真实模型的模型都相当有效的策略，即使该策略对于最大似然模型是次优的
 - i) 贝叶斯强化学习：它对关于正确模型的假设 h 赋予一个先验概率 $P(h)$ ，而后验概率 $P(h|e)$ 将在给定观测数据的情况下，通过贝叶斯法则得到。如果智能体在某一时刻决定停止学习，那么最优策略即为给出最高期望效用的策略。
 - ii) 健壮控制理论：考虑一组可能模型 H 而不赋予它们概率，在此基础上，最优健壮策略定义为能在 H 中最坏的情况下给出最佳结果的策略。
 - iii) 最坏情况假设：这一想法的问题在于，它会导致行为过于保守。如果一辆自动驾驶的汽车认为其他所有的司机都可能与它相撞，那它别无选择，只能停在车库里。现实生活中充满了这样的风险-回报权衡。

主动强化学习

时序差分 Q-学习

时序差分学习智能体要求智能体**必须学习转移模型**，以便它可以通过一步前瞻来选择基于 $U(s)$ 的动作

- **Q-学习方法**将通过学习**动作效用函数** $Q(s, a)$ 而不是学习效用函数 $U(s)$ 来避免对模型本身的需求。
 - $Q(s, a)$ 表示智能体在状态 s 下采取动作 a 并在其后采取最优行动的期望总折扣奖励。如果智能体已经知道 Q 函数，那么它可以通过简单地选择 $\arg\max_a Q(s, a)$ 来实现最优行动，而不需要基于转移模型进行前瞻探索
- 对于 Q 值的无模型TD更新规则

$$Q(s, a) = \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma \max_{a'} Q(s', a')]$$
$$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a, s') + \gamma \max_{a'} Q(s', a') - Q(s, a)].$$

- 每当智能体在状态 s 下执行动作 a 并导致状态 s' 时，就会计算此更新
- 无论在学习过程中还是动作选择中，时序差分 Q 学习智能体都**不需要转移模型** $P(s' | s, a)$

主动强化学习

时序差分 Q-学习

- **SARSA** (state, action, reward, state, action)
 - 即状态 (state)、动作 (action)、奖励 (reward)、状态 (state)、动作 (action)
 - 更新规则也与Q学习的更新规则非常相似，区别在于SARSA使用实际采取的动作 a' 的Q值进行更新，Q学习使用的是状态 s' 的最佳动作的 Q 值

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a, s') + \gamma Q(s', a') - Q(s, a)],$$

- Q学习是一种 **离策略 (off-policy)** 学习算法：所学习的 Q 值回答了问题 “假设停止使用正在使用的任何策略，并依照（根据估计）选择最佳动作的策略开始行动，那么这个动作在该状态下的 价值是多少？”
- SARSA是一个 **同策略 (on-policy)** 的算法，它通过学习 Q 值回答了问题 “假设坚持自己的策略，那么这个动作在该状态下的价值是多少？”

主动强化学习

```
function Q-LEARNING-AGENT(percept) returns an action
  inputs: percept, a percept indicating the current state  $s'$  and reward signal  $r$ 
  persistent:  $Q$ , a table of action values indexed by state and action, initially zero
                $N_{sa}$ , a table of frequencies for state–action pairs, initially zero
                $s, a$ , the previous state and action, initially null

  if  $s$  is not null then
    increment  $N_{sa}[s, a]$ 
     $Q[s, a] \leftarrow Q[s, a] + \alpha(N_{sa}[s, a])(r + \gamma \max_{a'} Q[s', a'] - Q[s, a])$ 
     $s, a \leftarrow s', \operatorname{argmax}_{a'} f(Q[s', a'], N_{sa}[s', a'])$ 
  return  $a$ 
```

一个探索Q学习智能体。它是一个主动的学习器，学习每个情境下每个动作的 $Q(s, a)$ 值。它采用了与探索ADP智能体所用相同的探索函数 f ，但是避免了对转移模型的学习

强化学习中的泛化

近似直接效用估计

- 在有更多状态的现实环境中, 不可能为了学习而简单地访问每一个状态
- 评价函数: 对想了解的潜在巨大状态空间的一个紧度量——把评价函数作为一个近似效用函数, 用函数近似来表示构造一个关于真实效用函数或Q函数的紧的近似的过程,

例, 加权线性组合近似: $\hat{U}_{\theta}(s) = \theta_1 f_1(s) + \theta_2 f_2(s) + \dots + \theta_n f_n(s)$

- 直接效用估计在状态空间中生成轨迹, 并对每个状态获取从该状态出发直到终止的奖励总和。状态和获得的奖励之和构成了监督学习算法的训练样例。
- 假设用一个简单的线性函数来表示4×3世界的效用函数, 其中方格的特征是它们的x和y坐标。在这种情况下有

$$\hat{U}_{\theta}(x, y) = \theta_0 + \theta_1 x + \theta_2 y.$$

- 在强化学习中使用在线学习算法可以使每次试验后的参数更新更有意义。
- Widrow-Hoff法则, 或delta法则, 对于在线最小二乘法: 为了使参数朝着误差减小的方向移动进行如下更新

$$\theta_i \leftarrow \theta_i - \alpha \frac{\partial E_j(s)}{\partial \theta_i} = \theta_i + \alpha [u_j(s) - \hat{U}_{\theta}(s)] \frac{\partial \hat{U}_{\theta}(s)}{\partial \theta_i}.$$

- $u_j(s)$: 在第j次试验中从状态s出发观测到的总奖励, 那么误差就定义为预测总奖励和实际总奖励的平方差的一半, 即: $E_j(s) = (\hat{U}_{\theta}(s) - u_j(s))^2/2$. 每个参数 θ_i 关于误差的变化率为、 $\partial E_j / \partial \theta_i$
- 改变参数 θ_i 以响应观测到的两个状态之间的转移也会改变其他状态的 $\hat{U}_{\theta}$ 值

强化学习中的泛化

近似时序差分

时序差分公式和Q学习公式的更新规则的新版本如下：

效用函数版本:

$$\theta_i \leftarrow \theta_i + \alpha [R(s, a, s') + \gamma \hat{U}_\theta(s') - \hat{U}_\theta(s)] \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i}$$

Q 值版本:

$$\theta_i \leftarrow \theta_i + \alpha [R(s, a, s') + \gamma \max_{a'} \hat{Q}_\theta(s', a') - \hat{Q}_\theta(s, a)] \frac{\partial \hat{Q}_\theta(s, a)}{\partial \theta_i}$$

- 对于被动TD学习，当函数近似器关于特征呈**线性**时，更新法则将收敛到最接近真实函数的近似。
- 在主动学习以及诸如神经网络等**非线性函数**中：即使假设空间中存在很好的解，**参数也将趋向无穷大**。
- **灾难性遗忘**
 - 学得太好了:忘记了早期的训练.价值函数变平坦，价值函数的二次特征的权重为零
 - 当价值函数使用有限维特征逼近时，持续在局部区域训练会使模型遗忘低频区域的价值结构，从而导致策略在未访问区域出现剧烈、灾难性的行为。
- 解决方案：**经验回放**
 - 保留整个学习过程中出现的轨迹，并对这些轨迹进行回放，以确保它不再访问的那部分状态空间上的价值函数仍然是准确的。

强化学习中的泛化

深度强化学习(Deep RL)

- 寻找性能超越线性函数的近似器
 - 对某些效用函数或Q函数来说, 可能不存在近似效果较好的线性函数
 - 无法找到需要的特征, 尤其是在一个新的领域中, 把U或Q表示为特征的线性组合总是可能的, 尤其是当我们有形如 $f_1(s) = U(s)$ 或 $f_2(s, a) = Q(s, a)$ 的特征时, 但是除非能提供这样的特征 (以有效的可计算形式), 否则线性函数近似器可能是不够的。
- 需要复杂的非线性函数近似器
- 深度强化学习系统: 使用深度网络作为函数近似器的强化学习系统
- Deep RL 通常很难有良好的表现, 并且如果训练数据与真实环境稍有不同, 训练系统的行为可能会变得非常不可预测

奖励函数设计

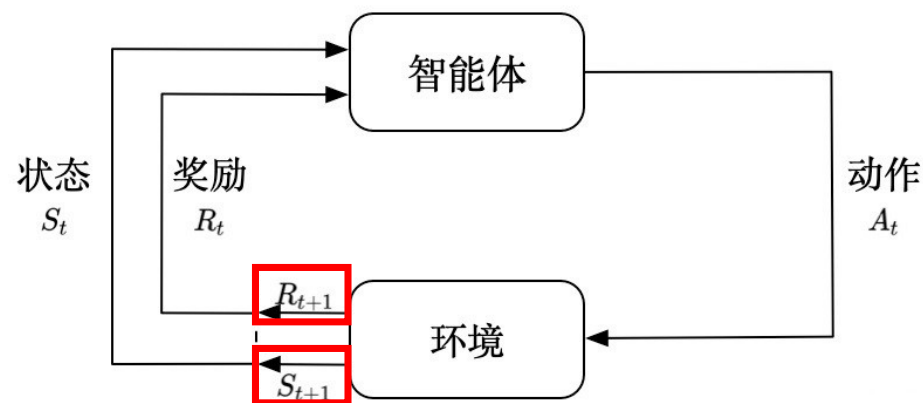
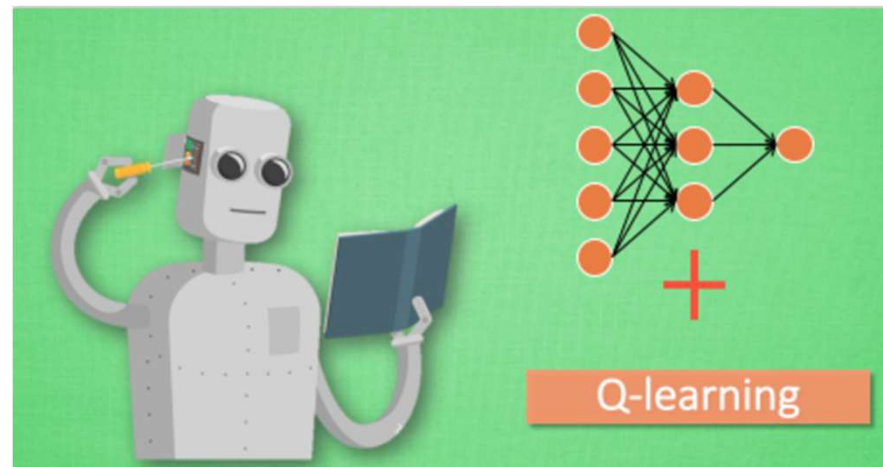
- 信用分配问题: 找出智能体哪一步做错了
- 奖励函数设计为智能体提供额外的奖励, 称之为伪奖励 (pseudoreward), 用于奖励智能体 “取得进步”

$$R'(s, a, s') = R(s, a, s') + \gamma \Phi(s') - \Phi(s)$$

深度强化学习（Deep Reinforcement Learning, DRL）基本概念

强化学习与神经网络

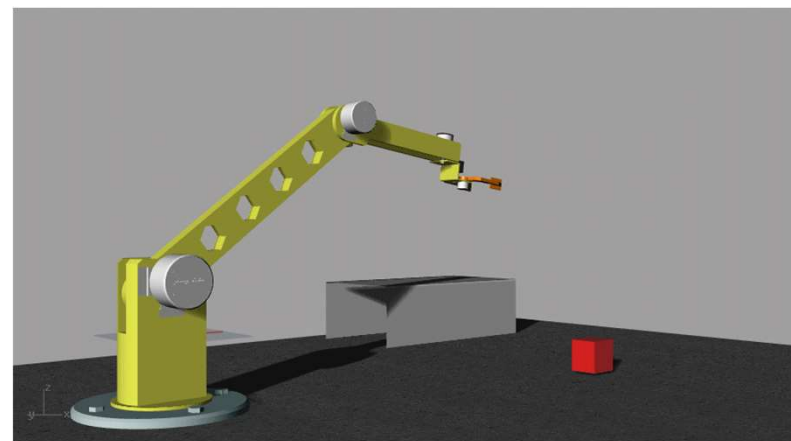
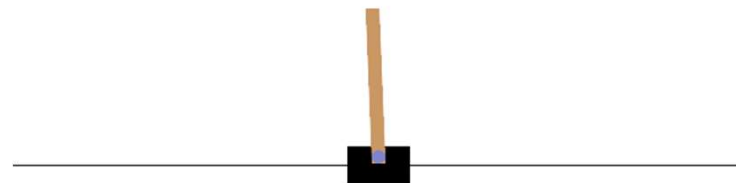
- Q-learning、Sarsa算法等强化学习方法都是比较传统的强化学习算法，随着机器学习在日常生活中的各种应用，各种机器学习方法也在融汇合并.
- 足够深度的神经网络能够拟合任意函数（通用近似定理），强化学习中的策略函数和值函数也可以通过**神经网络拟合**. 同时，深度学习中的梯度随机下降算法也适用于拟合的神经网络.



深度强化学习（Deep Reinforcement Learning, DRL）基本概念

深度学习模型的引入

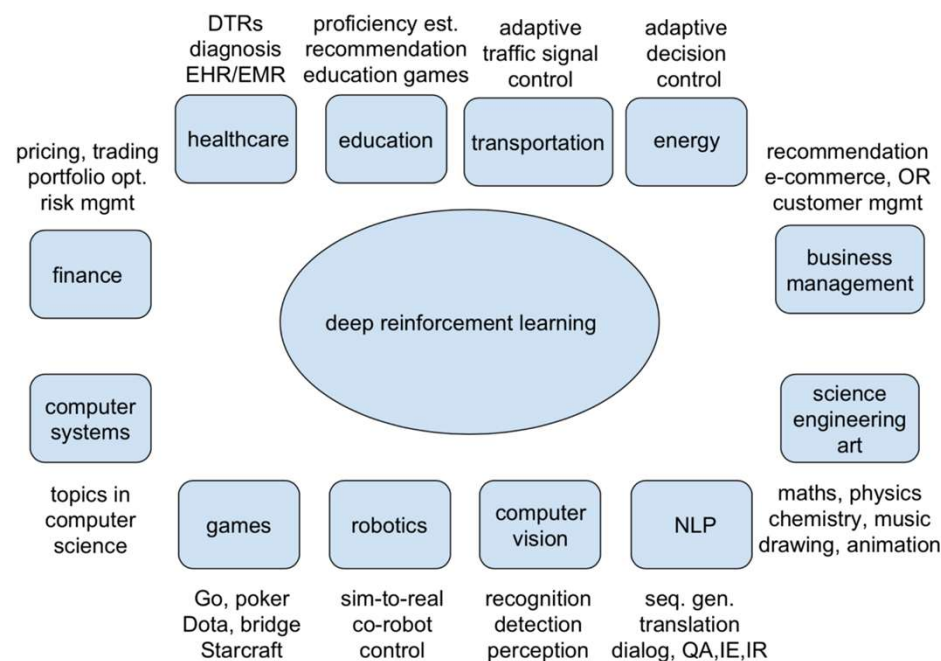
- 在深度学习中，根据模型拟合的数据类型可以把模型分为**分类(Categorical)模型**和**回归(Regression)模型**两种。同样，深度强化学习的模型根据对应的输出也可以分为离散的和连续的两种。对应的策略网络和价值网络的输出也有离散的和连续的两种，我们需要据实际问题来动态选择网络的输出层是离散的还是连续的。
- 典型的**离散**的强化学习模型可以应用的环境包括经典的倒立摆环境，对应的控制是离散的左右方向，通过制定每一时刻滑块的运动方向来稳定倒立摆。
- 典型的**连续**强化学习模型应用的环境包括机械臂的控制等，对应的机械臂的参数能够动态地在一定范围内变化，这时就需要深度学习模型能够处理连续的值。



深度强化学习（Deep Reinforcement Learning, DRL）基本概念

深度学习模型的引入

- 除了以上概念，深度强化学习和传统强化学习最大的区别在于**各种深度学习模型**的引入。由于结合了深度学习，强化学习已经不但可以处理一些简单的输入数值，更重要的是，强化学习能够处理复杂的数据，比如图像和文本等。
- 通过结合对应的深度学习模型(比如深度卷积网络)，深度强化学习能够从复杂的输入(这些输入对应智能体的状态)提取对应状态的特征，并且根据状态的特征来做出合理的决策(策略网络)，或是估计当前状态的价值(价值网络)。
- 由于深度学习使用的模型具有较大的参数，可以通过这些模型来更加精准地拟合对应的函数，这样又大大提高了算法的效率。通过在强化学习中引入深度学习，可以说同时扩展了强化学习算法的应用边界和效率。



Yuxi Li, Deep Reinforcement Learning, arXiv, 2018

深度强化学习 (Deep Reinforcement Learning, DRL) 基本概念

传统强化学习方法

传统RL通常基于较简单的模型(如右图中Q-learning、SARSA等), 并且在求解TSP问题等路径规划问题时, 依赖于手工设计的特征和启发式策略.

优点:

- 简单性: 算法结构清晰;
- 无需大规模数据: 不需要大量的训练数据和复杂的计算资源;
- 可解释性强: 训练过程的可解释性较强.

缺点:

- 状态空间限制: 状态空间随着问题规模的增大呈指数级增长;
- 探索效率低: 大规模问题训练速度较慢, 效率较低;
- 难以捕捉复杂模式: 缺乏强大的特征提取能力.

Q-Learning

Off-policy

```
Initialize  $Q(s, a)$  arbitrarily
Repeat (for each episode):
  Initialize  $s$ 
  Repeat (for each step of episode):
    Choose  $a$  from  $s$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
    Take action  $a$ , observe  $r, s'$ 
     $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ 
     $s \leftarrow s'$ 
  until  $s$  is terminal
```

Sarsa

On-policy

```
Initialize  $Q(s, a)$  arbitrarily
Repeat (for each episode):
  Initialize  $s$ 
  Choose  $a$  from  $s$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
  Repeat (for each step of episode):
    Take action  $a$ , observe  $r, s'$ 
    Choose  $a'$  from  $s'$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
     $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$ 
     $s \leftarrow s'; a \leftarrow a'$ 
  until  $s$  is terminal
```

深度强化学习 (Deep Reinforcement Learning, DRL) 基本概念

深度强化学习方法

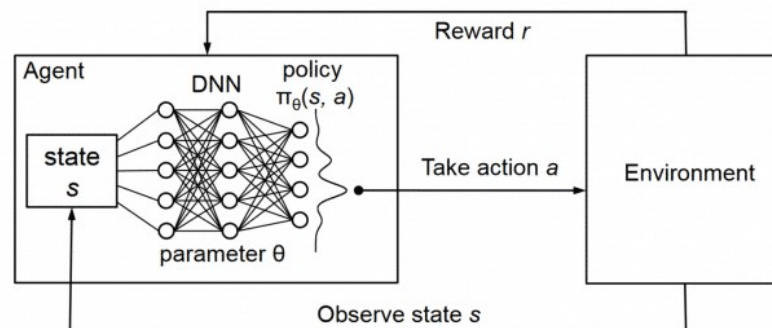
DRL通过引入深度神经网络,尤其是在图结构数据(如TSP)中,能够自动学习复杂的特征表示,并提高在大规模问题中的处理能力.

优点:

- 高维状态空间处理: DRL能够处理复杂的状态和动作空间;
- 自动学习特征: 使用深度神经网络(如卷积神经网络、图神经网络、Transformer等);
- 较好的泛化能力: DRL能够在不同规模的问题上展示较好的泛化能力,避免了针对具体问题过拟合;
- 端到端优化: 减少复杂性.

缺点:

- 训练成本高: 需要大量计算资源和时间;
- 训练不稳定: 梯度爆炸或梯度消失;
- 解的质量不稳定: 不保证能够找到最优解.



Algorithm 1: deep Q-learning with experience replay.

```
Initialize replay memory  $D$  to capacity  $N$ 
Initialize action-value function  $Q$  with random weights  $\theta$ 
Initialize target action-value function  $\hat{Q}$  with weights  $\theta^- = \theta$ 
For episode = 1,  $M$  do
    Initialize sequence  $s_1 = \{x_1\}$  and preprocessed sequence  $\phi_1 = \phi(s_1)$ 
    For  $t = 1, T$  do
        With probability  $\epsilon$  select a random action  $a_t$ 
        otherwise select  $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$ 
        Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
        Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
        Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $D$ 
        Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $D$ 
        Set  $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$ 
        Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  with respect to the network parameters  $\theta$ 
        Every  $C$  steps reset  $\hat{Q} = Q$ 
    End For
End For
```

深度强化学习（Deep Reinforcement Learning, DRL）基本概念

深度强化学习方法

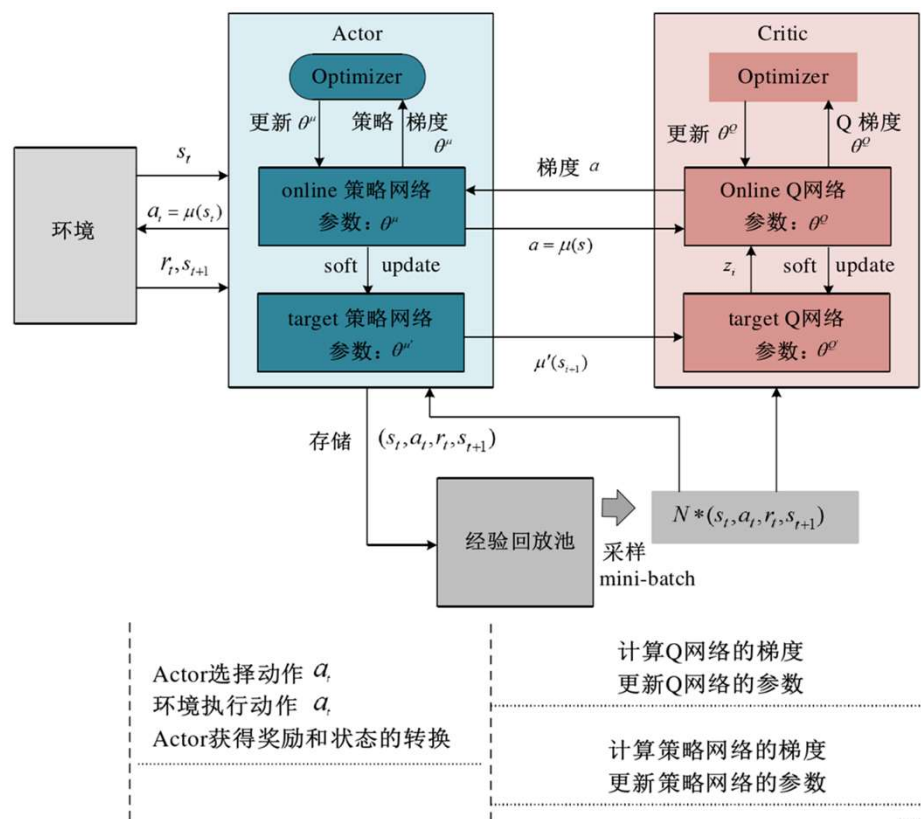
DRL通过引入深度神经网络，尤其是在图结构数据(如TSP)中，能够自动学习复杂的特征表示，并提高在大规模问题中的处理能力.

优点:

- 高维状态空间处理：DRL能够处理复杂的状态和动作空间；
- 自动学习特征：使用深度神经网络(如卷积神经网络、图神经网络、Transformer等)；
- 较好的泛化能力：DRL能够在不同规模的问题上展示较好的泛化能力，避免了针对具体问题过拟合；
- 端到端优化：减少复杂性.

缺点:

- 训练成本高：需要大量计算资源和时间；
- 训练不稳定：梯度爆炸或梯度消失；
- 解的质量不稳定：不保证能够找到最优解.



深度强化学习（Deep Reinforcement Learning, DRL）基本概念

特性	传统强化学习（RL）	深度强化学习（DRL）
适用问题规模	适用小到中规模问题	适用中到大规模问题
状态空间处理	需要手动设计状态表示	能够自动从数据中学习高维特征表示
计算资源要求	计算资源相对较少	需要大量的计算资源（GPU、TPU等）
训练数据量要求	相对较少，样本效率较高	需要大量的训练数据和探索
解的质量	解的质量较为稳定，尤其在小规模时较好	解的质量可能波动，尤其在大规模问题时
泛化能力	受限于手动设计的状态和动作空间	较强的泛化能力，能够适应不同规模的TSP问题
训练速度	快，适用于小规模TSP	慢，尤其在处理大规模TSP时
复杂性	相对简单，易于实现	相对复杂，需要深度学习框架支持

强化学习中的泛化

分层强化学习

- 分层强化学习 (HRL): 处理过长动作序列时, 把它们分成几个小片段, 然后再把它们分成更小的片段, 并以此类推, 直到动作序列足够短, 进而使学习变得容易
- 一个局部程序 (partial program) 出发, 该程序刻画了智能体行为的某种分层结构
- 智能体程序的局部编程语言通过为必须通过学习得到的未指定的选择添加主程序的方式扩展了原始的编程语言
- HRL的理论基础建立在联合状态空间 (joint state space) 的概念之上, 该状态空间的每个状态 (s, m) 由一个物理状态 s 和一个机器状态 m 组成
- 一个选择状态 $\sigma = (s, m)$ 是指 m 的程序计数器正处于智能体程序中的一个选择点的状态。在两个选择状态之间, 可能会发生任意数量的转移和物理动作, 但它们都是预先注定的。
- 从本质上讲, 分层强化学习智能体求解的是一个马尔可夫决策问题, 它包含以下元素:
 - 状态为联合状态空间的选择状态 σ 。
 - 在状态 σ 采取的动作是由局部程序决定的状态 σ 上可行的选择 c
 - 奖励函数 $\rho(\sigma, c, \sigma')$ 是在选择状态 σ 和 σ' 之间发生的物理转移的期望奖励总和
 - 转移模型 $\tau(\sigma, c, \sigma')$ 将通过一种显式的方法定义: 如果 c 采用了一个物理动作 a , 那么 τ 将来自物理模型 $P(s'|s, a)$; 如果 c 采用了一个计算转移, 例如调用了一个子程序, 那么转移将根据编程语言规则确定性地改变计算状态 m 。

策略搜索

- 核心思想:只要策略的表现有所改进, 就继续调整策略, 直到停止
- 策略 π 是一个将状态映射到动作的函数, 主要感兴趣的是 π 的**参数化表示**, 它的参数比状态空间中的状态少得多。
 - 例: 可以用一组参数化的Q函数来表示 π , 每个函数对应一个动作, 然后选取预测值最高的动作 (**这个过程不同于Q学习!**)

$$\pi(s) = \operatorname{argmax}_a \hat{Q}_\theta(s, a)$$

- 问题: 即如果动作是**离散**的, 策略将是关于参数的**不连续函数**。存在某些 θ 值, 使得 θ 本身的微小变化会导致策略从一个动作切换到另一个动作。这意味着策略的价值也可能随着参数不连续地改变, 这使得基于梯度的搜索变得困难。

- 随机策略表示 $\pi(s, a)$, 指定了在 s 状态下选择动作 a 的概率。一种较为常用的表示是softmax函数:

$$\pi_\theta(s, a) = \frac{e^{\beta \hat{Q}_\theta(s, a)}}{\sum_{a'} e^{\beta \hat{Q}_\theta(s, a')}}.$$

- 策略梯度向量 $\nabla_\theta \rho(\theta)$, 在 $\rho(\theta)$ 可微的前提下.

$$\nabla_\theta \rho(\theta) \approx \frac{1}{N} \sum_{j=1}^N \frac{u_j(s) \nabla_\theta \pi_\theta(s, a_j)}{\pi_\theta(s, a_j)}$$

学徒学习与逆强化学习

学徒学习 (apprenticeship learning) 研究这样的问题：在提供了一些对专家的行为观测的基础上，我们如何让学习表现得较好

- 模仿学习: 假设环境是可观测的，对观测到的状态-动作对应用监督学习方法以学习策略 $\pi(s)$.
 - **局限**: 训练集中的微小误差将随着时间累积增长，并最终导致学习失败。并且模仿学习最多只能复现教师的表现，而不能超越教师的表现。)
- 逆强化学习 (IRL): 通过观察策略来学习奖励，而不是通过观察奖励来学习策略
 - 旨在理解原因: 观察专家的行为（和结果状态），并试图找出专家所最大化的奖励函数，进而得到一个关于这个奖励函数的最优策略
 - 特征匹配: 假设奖励函数可以写成特征的加权线性组合

$$R_{\theta}(s, a, s') = \sum_{i=1}^n \theta_i f_i(s, a, s') = \theta \cdot \mathbf{f}.$$

强化学习的应用

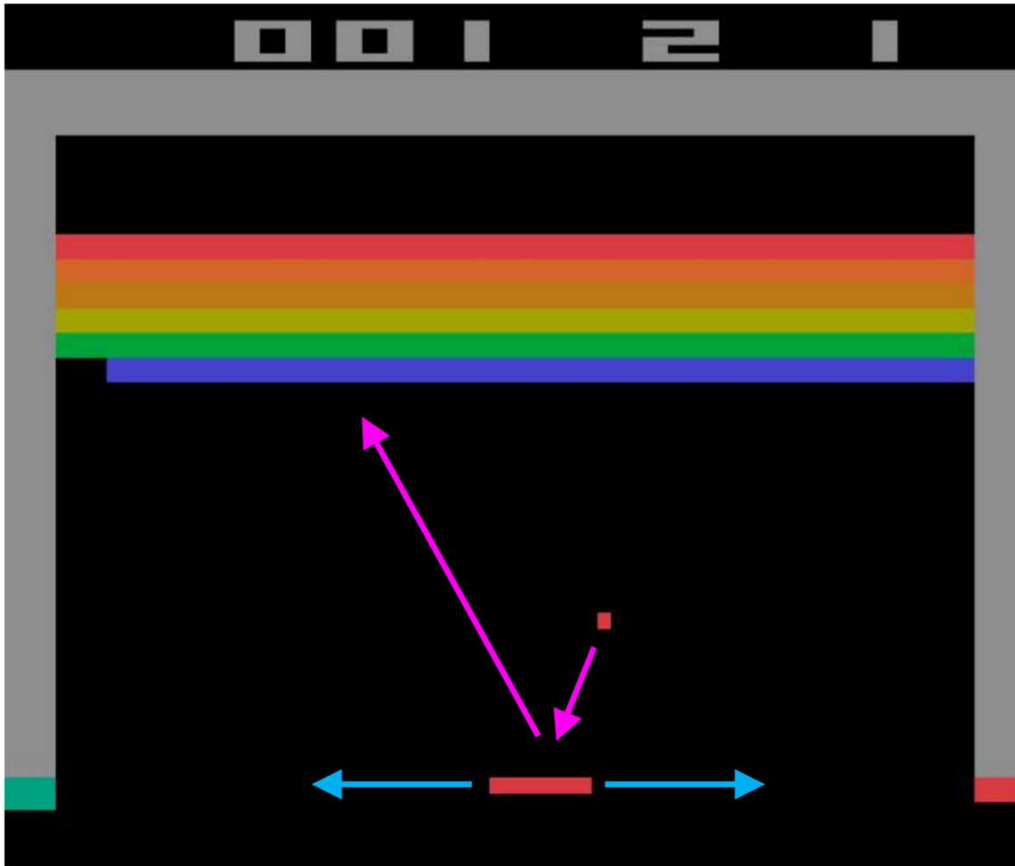
在电子游戏中的应用

- 深度Q网络 (DQN), 来自DeepMind的一个团队开发了第一个现代深度强化学习系统
- DQN采用深度神经网络表示Q函数, 除此之外它其实是典型的强化学习系统
- DQN已经分别在49款不同的Atari电子游戏中训练过。
- 学会了模拟赛车、射击外星人的宇宙飞船以及用桨弹球
- Al p h a Go 使用深度强化学习在围棋比赛中击败了最高水平的人类选手

在机器人控制中的应用

- 车杆平衡问题, 即倒立摆问题, 需要通过施加外力使手推车左右摇摆来使得摆杆大致保持竖直状态 ($\theta=90^\circ$), 同时也需要保持位置x在轨道的限制范围内。
- 无线电控制直升机飞行
 - 通常在大型MDP上使用策略搜索来完成
 - 通常与模仿学习以及对人类专家飞行员进行观测的逆强化学习相结合

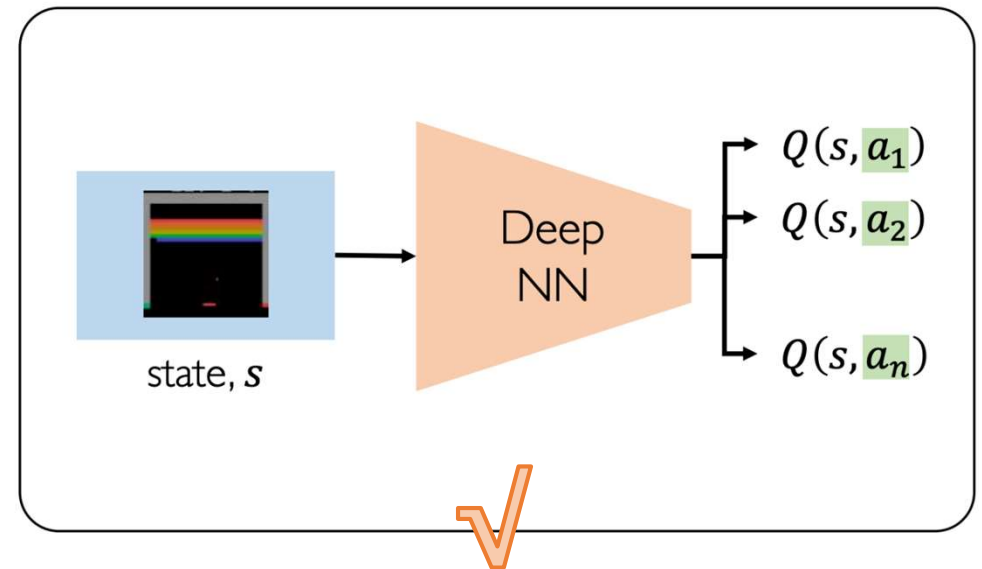
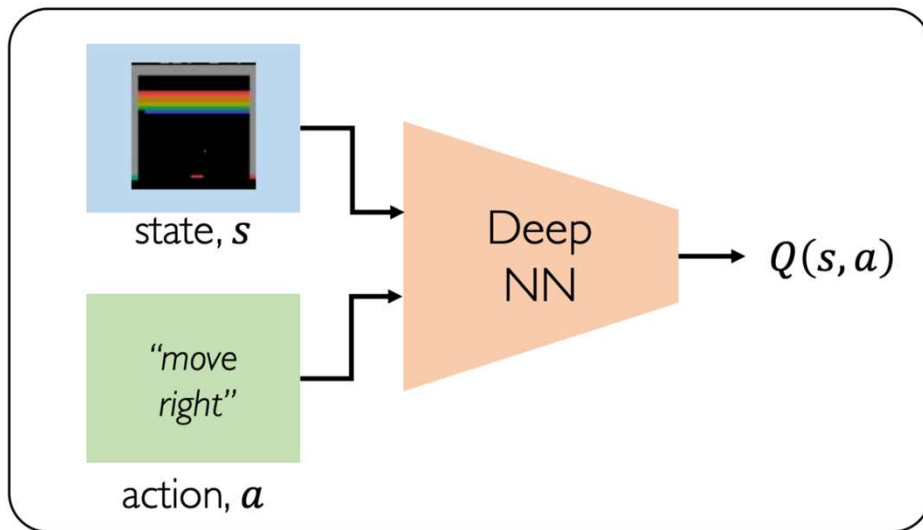
DQN for Atari Games



特点

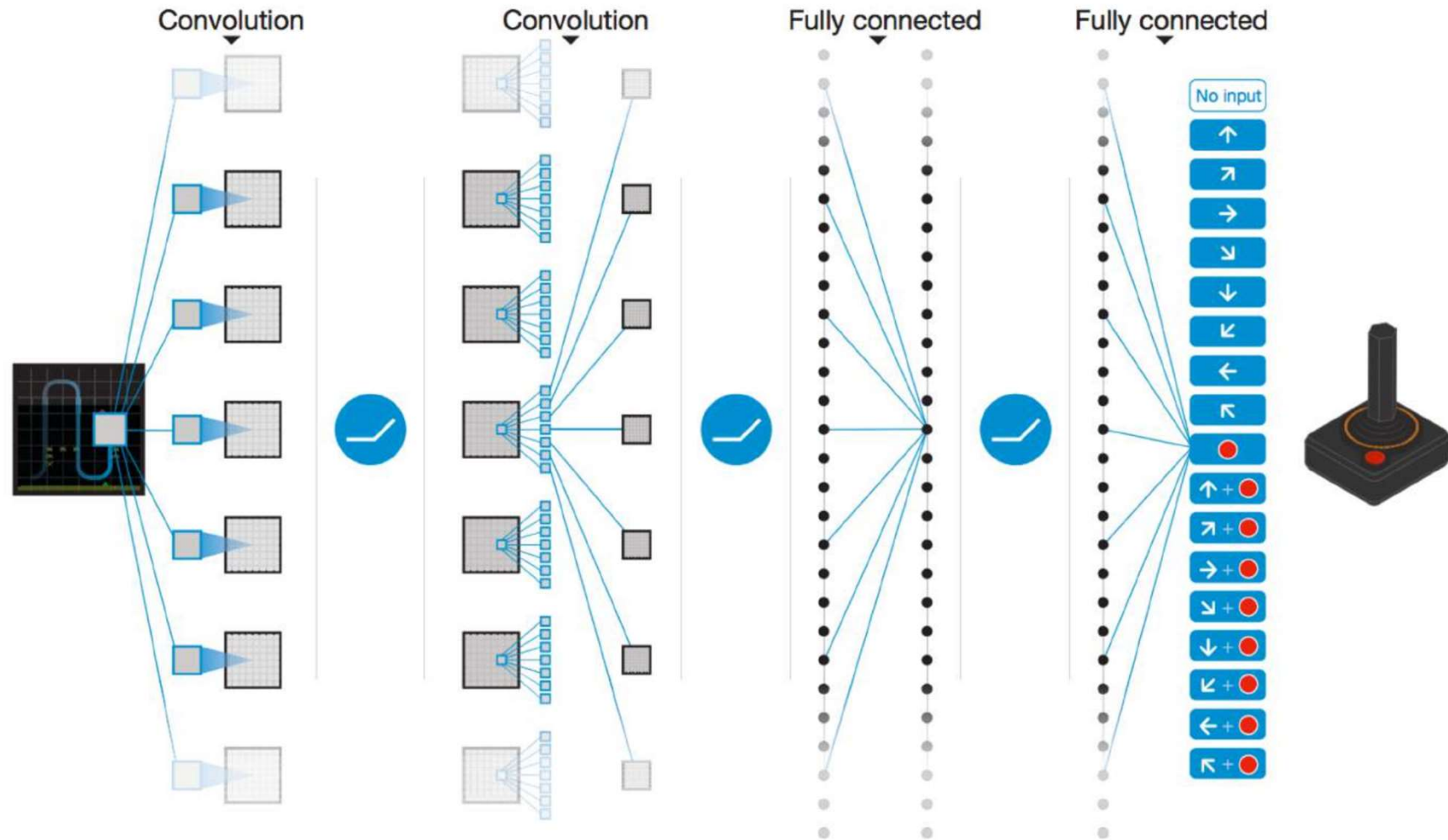
1. 从像素输入直接决策
2. 动作空间小但动态复杂
3. 奖励稀疏
4. 任务多样性强
5. 标准化评测平台

DQN for Atari Games

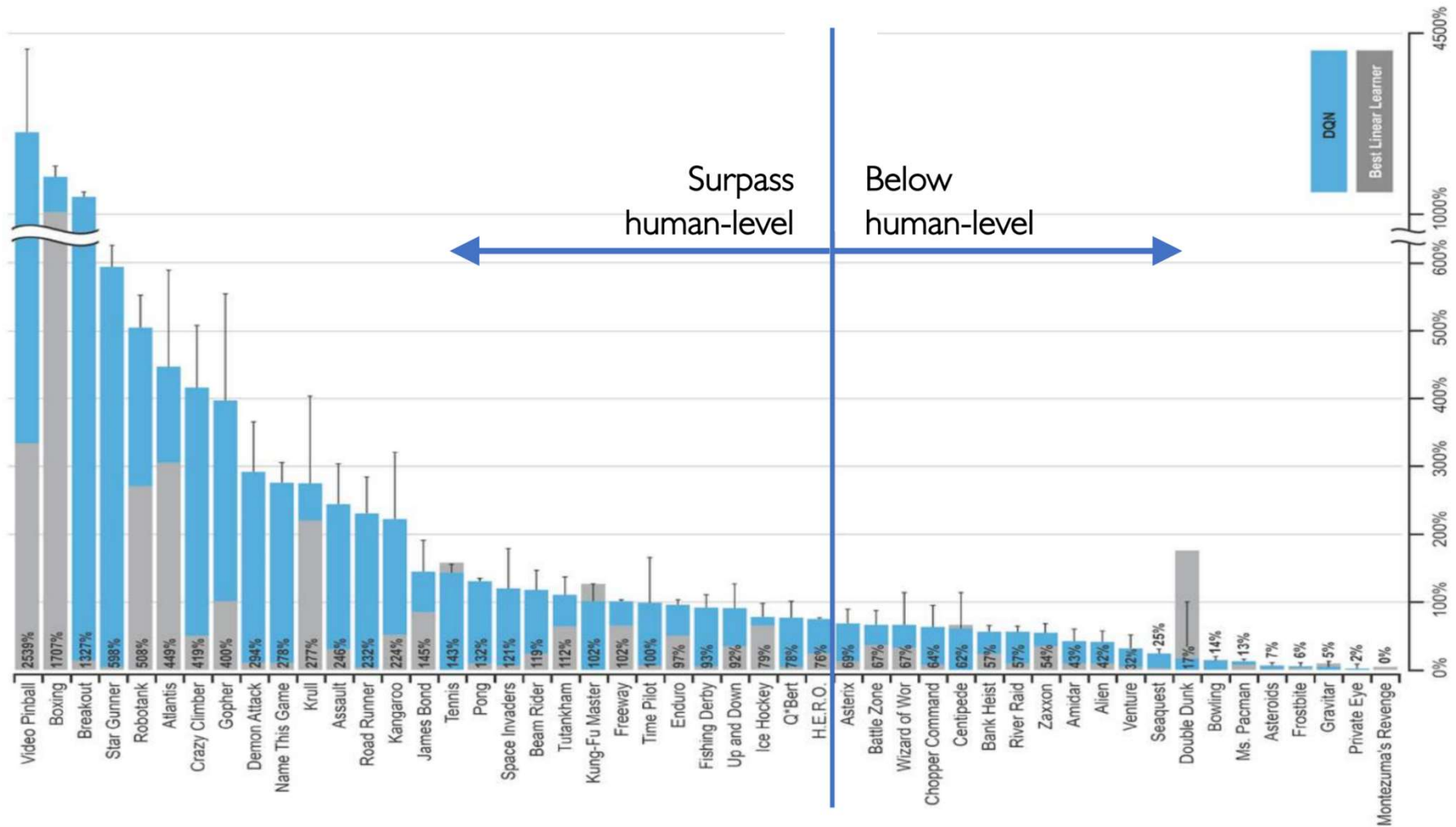
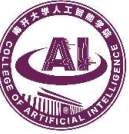


$$\mathcal{L} = \mathbb{E} \left[\left\| \left(r + \gamma \max_{a'} Q(s', a') \right) - Q(s, a) \right\|^2 \right]$$

DQN for Atari Games

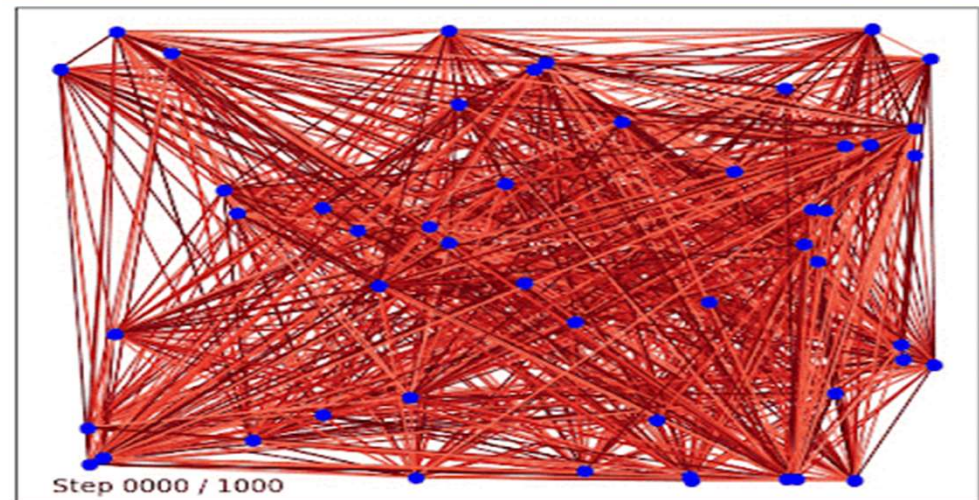


DQN for Atari Games



深度强化学习用于机器人规划的基本方法（以DQN为例）

- 以Q-learning为例，使用**Q表格**来存储每一个状态state, 和在这个state每个行为action所拥有的Q值.
- 当问题复杂度较高时, 所有的状态全用表格来存储, 超过了现有计算机可接受的内存上限, 并且在数据量过大的表格中搜索对应的状态也非常耗时.
- 将**状态和动作当成神经网络的输入**, 然后经过神经网络分析后得到动作的Q值, 不需要在表格中记录Q值, 而是直接使用神经网络生成Q值.
- 按照Q-learning的原则,**只输入状态值, 输出所有的动作值**, 直接选择拥有最大值的动作当做下一步要做的动作. 我们可以想象, 神经网络接受外部的信息, 相当于眼睛鼻子耳朵收集信息, 然后通过大脑加工输出每种动作的值, 最后通过强化学习的方式选择动作.



深度强化学习用于机器人规划的基本方法（以DQN为例）

- 以Q-learning为例, 使用**Q表格**来存储每一个状态state, 和在这个state每个行为action所拥有的Q值.
- 当问题复杂度较高时, 所有的状态全用表格来存储, 超过了现有计算机可接受的内存上限, 并且在数据量过大的表格中搜索对应的状态也非常耗时.
- 将**状态和动作当成神经网络的输入**, 然后经过神经网络分析后得到动作的Q值, 不需要在表格中记录Q值, 而是直接使用神经网络生成Q值.
- 按照Q-learning的原则,**只输入状态值, 输出所有的动作值**, 直接选择拥有最大值的动作当做下一步要做的动作. 我们可以想象, 神经网络接受外部的信息, 相当于眼睛鼻子耳朵收集信息, 然后通过大脑加工输出每种动作的值, 最后通过强化学习的方式选择动作.



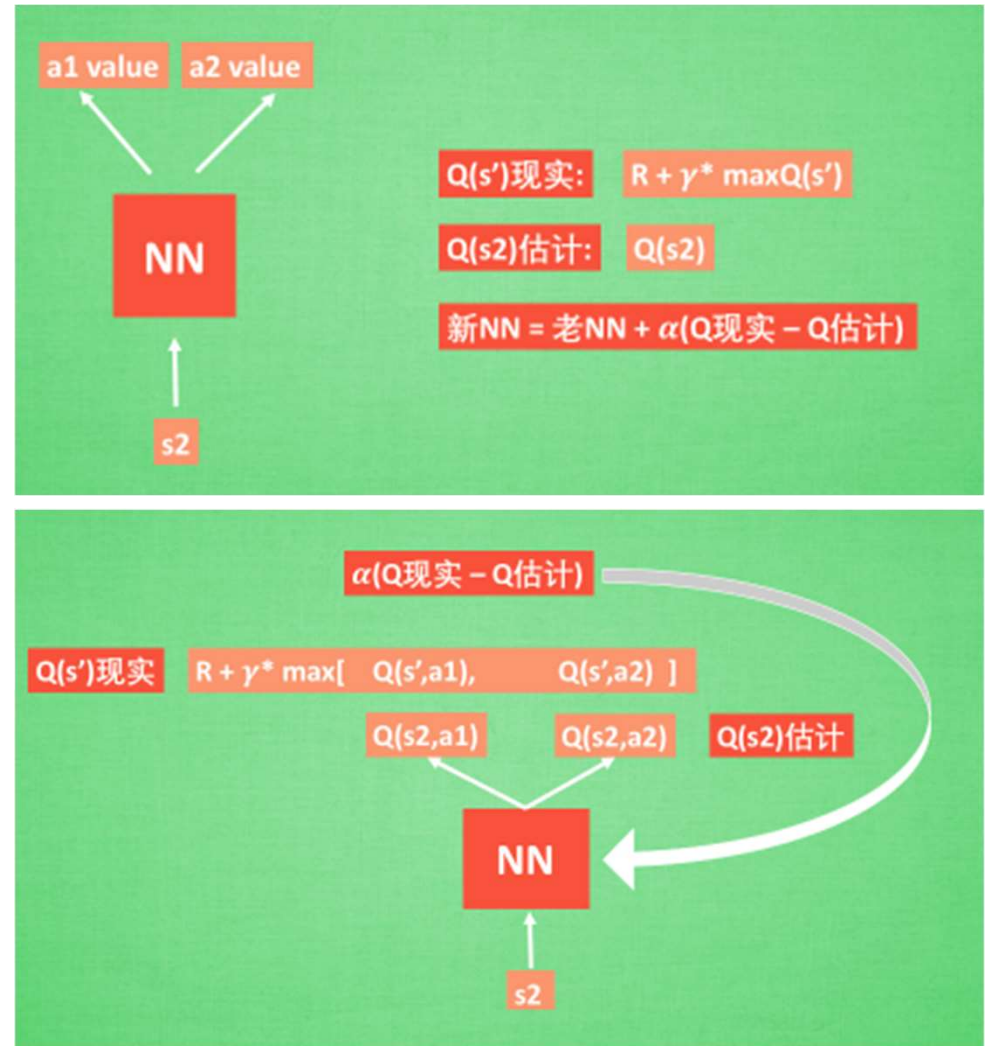
深度强化学习用于机器人规划的基本方法（以DQN为例）

- 以Q-learning为例，使用**Q表格**来存储每一个状态state, 和在这个state每个行为action所拥有的Q值.
- 当问题复杂度较高时, 所有的状态全用表格来存储, 超过了现有计算机可接受的内存上限, 并且在数据量过大的表格中搜索对应的状态也非常耗时.
- 将**状态和动作当成神经网络的输入**, 然后经过神经网络分析后得到动作的Q值, 不需要在表格中记录Q值, 而是直接使用神经网络生成Q值.
- 按照Q-learning的原则,**只输入状态值, 输出所有的动作值**, 直接选择拥有最大值的动作当做下一步要做的动作. 我们可以想象, 神经网络接受外部的信息, 相当于眼睛鼻子耳朵收集信息, 然后通过大脑加工输出每种动作的值, 最后通过强化学习的方式选择动作.



深度强化学习用于机器人规划的基本方法（以DQN为例）

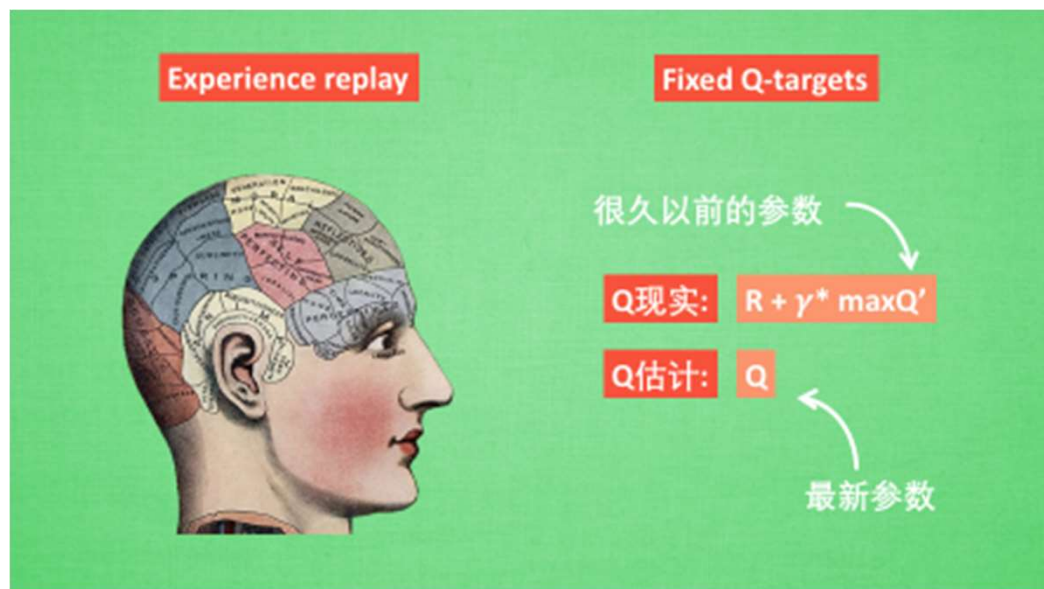
- 在强化学习中, 神经网络是如何被训练的呢? 首先, 我们需要 $a1, a2$ 正确的Q值, 这个Q值我们就用之前在Q-learning中的Q现实来代替. 同样我们还需要一个Q估计来实现神经网络的更新. 所以神经网络的参数就是老的 **NN 参数** 加**学习率 α** 乘以 **Q 现实** 和 **Q 估计** 的差距.
- 通过NN预测出 $Q(s2, a1)$ 和 $Q(s2, a2)$ 的值, 这就是Q估计. 然后我们选取Q估计中最大值的动作来换取环境中的奖励reward. 而Q现实中也包含从神经网络分析出来的两个Q估计值, 不过这个Q估计是针对于下一步在 s' 的估计. 最后再通过刚刚所说的算法更新神经网络中的参数. 还有两大因素支撑着DQN使得它变得无比强大. 这两大因素就是 **Experience replay** 和 **Fixed Q-targets**.



深度强化学习用于机器人规划的基本方法（以DQN为例）

DQN两大利器

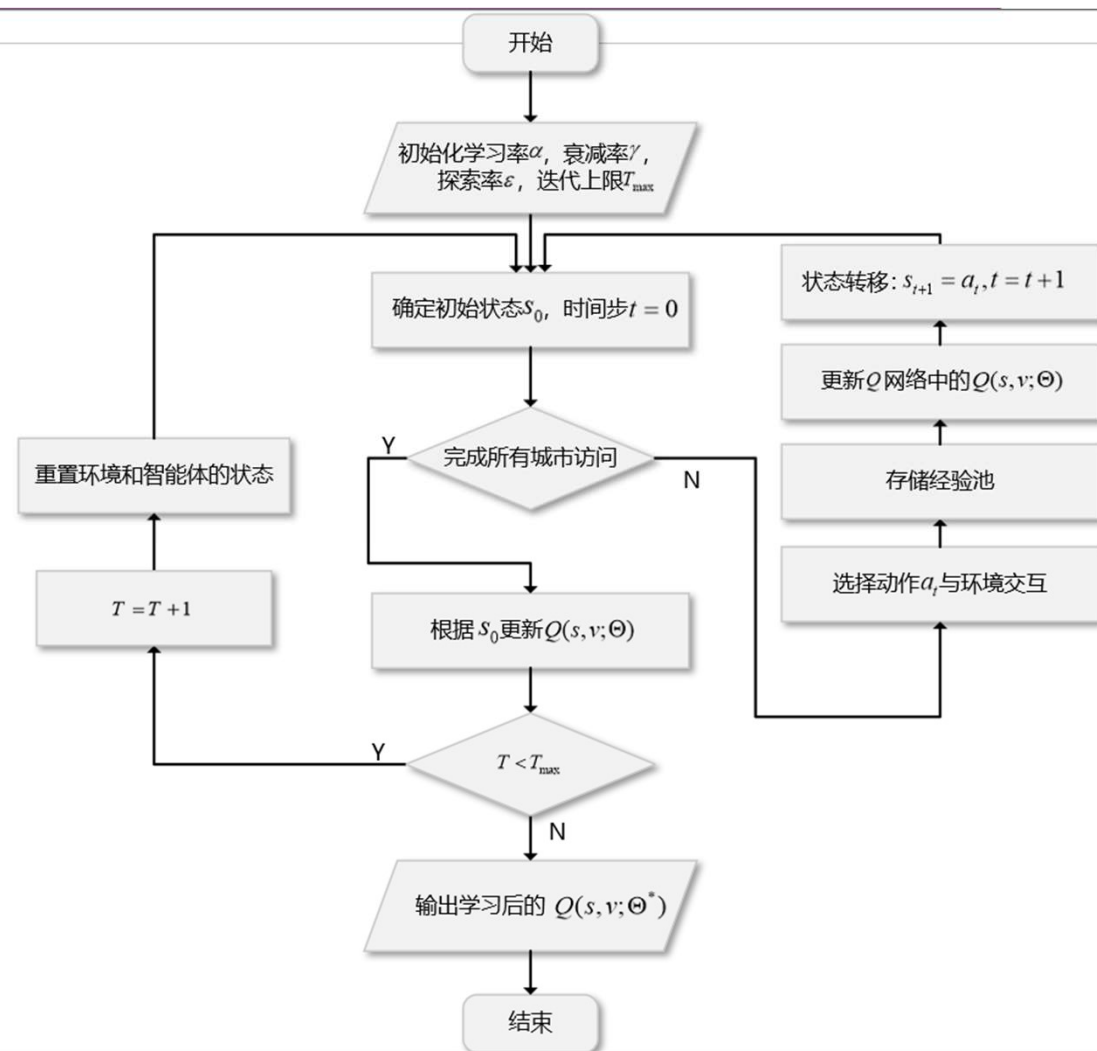
DQN 有一个记忆库用于**学习之前的经历**.与之相比, Q-learning是一种off-policy离线学习法, 它能学习当前经历着的, 也能学习过去经历过的, 甚至是学习别人的经历. 每次DQN更新的时候, 我们都可以随机抽取一些之前的经历进行学习. 随机抽取这种做法打乱了经历之间的相关性, 也使得神经网络更新更有效率. **Fixed Q-targets** 也是一种打乱相关性的机理, 如果使用 **Fixed Q-targets**, 我们就会在DQN中使用到两个结构相同但参数不同的神经网络, 预测Q估计的神经网络具备最新的参数, 而预测Q现实的神经网络使用的参数则是很久以前的.



深度强化学习用于机器人规划的基本方法（以DQN为例）

基于深度强化学习的TSP问题求解

构建一个名为 Q_func 的对象，它将代表我们的 $Q()$ 函数神经网络。然后，对于每一步，采样一个新的随机全连接图（理想情况下，从接近我们最终问题的分布中采样）。只要我们的解决方案不代表完整的旅程，就会根据 ϵ -贪婪策略选择下一个节点：以概率 ϵ ，我们随机选择一个新节点，以概率 $(1-\epsilon)$ ，我们选择最大化 $Q(s, a)$ 的节点（动作 a ）。我们将在训练过程中减少 ϵ ，直到达到某个最小探索概率，以便随着 $Q()$ 变得越来越准确，使算法越来越贪婪。



深度强化学习用于机器人规划的基本方法（以DQN为例）

基于深度强化学习的TSP问题求解

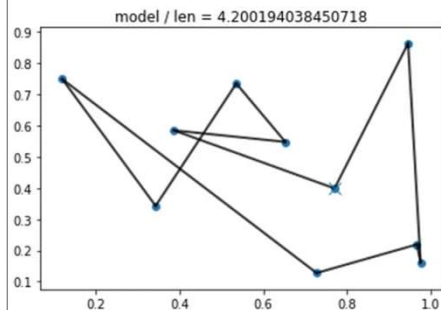
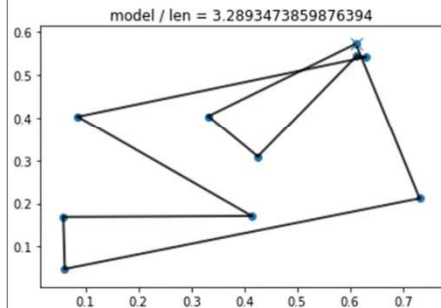
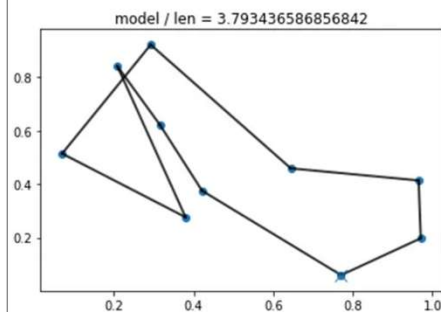
Q网络更新

$$\mu_v^{(t+1)} \leftarrow \text{relu}(\theta_1 x_v + \theta_2 \sum_{u \in \mathcal{N}(v)} \mu_u^{(t)} + \theta_3 \sum_{u \in \mathcal{N}(v)} \text{relu}(\theta_4 w(v, u)))$$

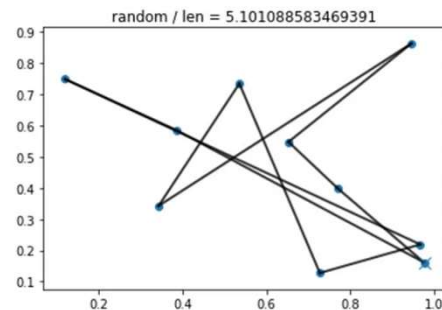
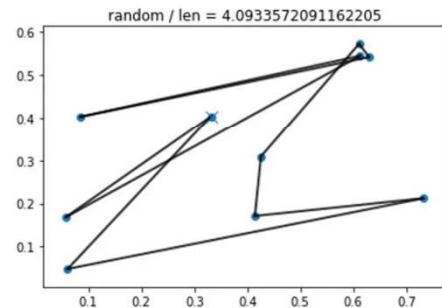
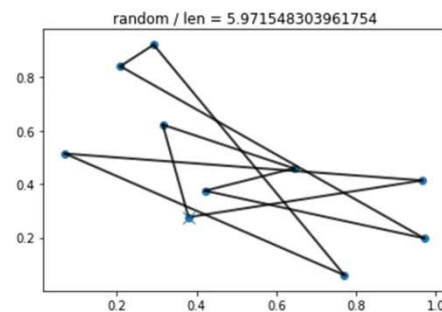
累积奖励估计

$$Q(s, v; \Theta) = \theta_5^T \text{relu} \left(\left[\theta_6 \sum_{u \in V} \mu_u^{(T)}, \theta_7 \mu_v^{(T)} \right] \right)$$

Agent



Random



强化学习用于求解旅行商问题的基本方法（以Q-learning为例）

TD算法

- 初始对目标的预期:

$$\hat{q} = Q(\text{北京}, \text{上海}; w) = 14$$

- 进行部分观测后对目标的预期:

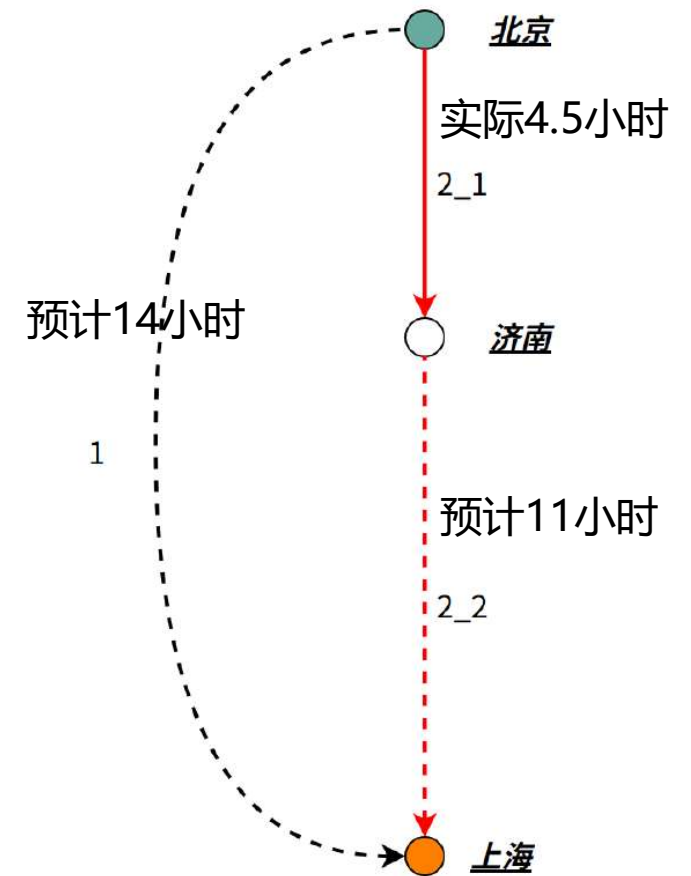
$$\hat{q}' \triangleq Q(\text{济南}, \text{上海}; w) = 11.$$

$$\hat{y} \triangleq r + \hat{q}' = 4.5 + 11 = 15.5.$$

- TD 误差:

$$\hat{q} - \hat{q}' = 14 - 11 = 3.$$

$$\delta = 3 - 4.5 = -1.5.$$



强化学习用于求解旅行商问题的基本方法（以Q-learning为例）

Q-Learning算法

- 当前状态的最优动作价值函数($\tilde{Q}$ 是估计, Q 是最优):

$$Q_a(s_t, a_t) = \max_{\pi} E[U_t | S_t = s_t, A_t = a_t]$$

$$= E_{S_{t+1} \sim p(\cdot | s_t, a_t)} [R_t + \gamma \cdot \max_{A \in A} Q(S_{t+1}, A) | S_t = s_t, A_t = a_t].$$

- 进行部分观测后对目标的预期(用 $\tilde{Q}$ 代替 Q):

$$\hat{y}_t \triangleq r_t + \gamma \cdot \max_{a \in A} \tilde{Q}(s_{t+1}, a).$$

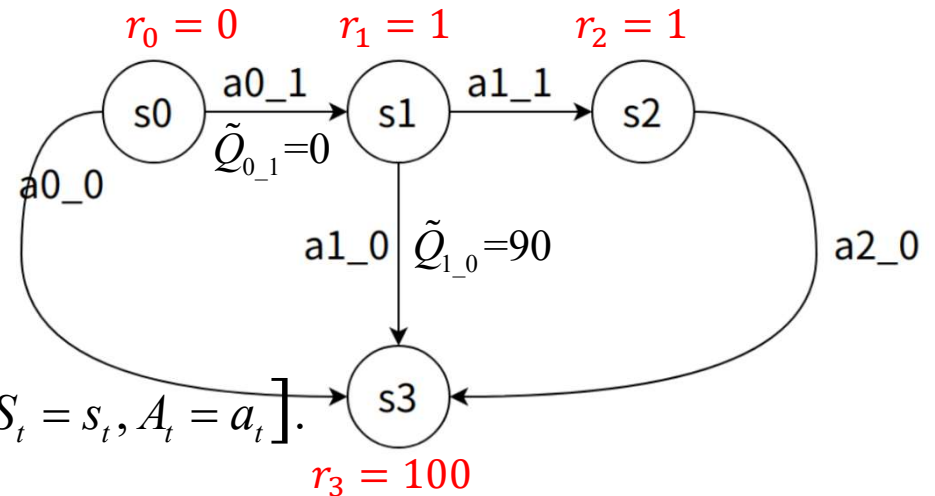
$$\hat{y} \triangleq r + \hat{q}' = 4.5 + 11 = 15.5.$$

- 用部分观测的 $\hat{y}_t$ 更新 Q :

$$\tilde{Q}(s_t, a_t) \leftarrow (1 - \alpha) \cdot \tilde{Q}(s_t, a_t) + \alpha \cdot \hat{y}_t.$$

- 贝尔曼方程

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R + \gamma \max_{A \in A} Q_?(S_{t+1}, A) - Q(S_t, A_t)]$$



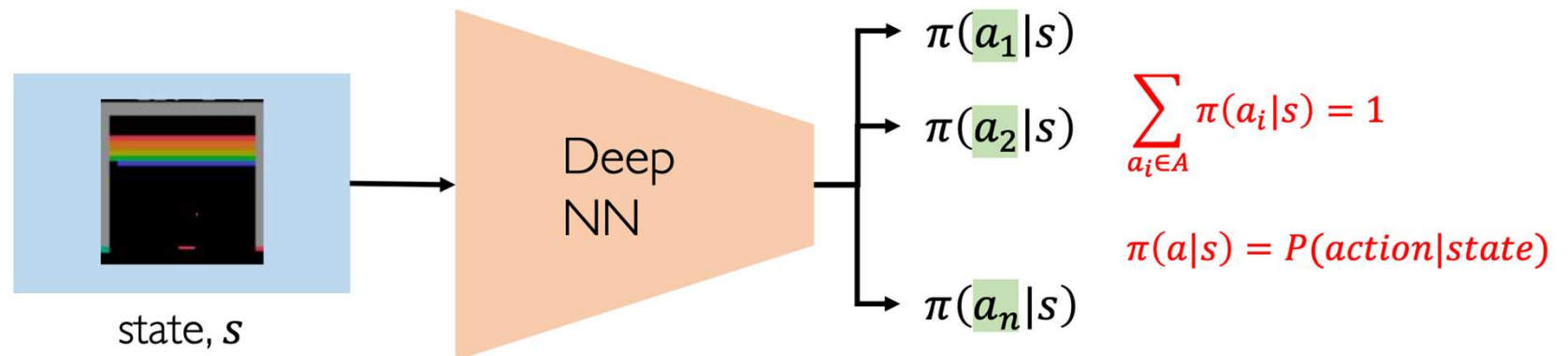
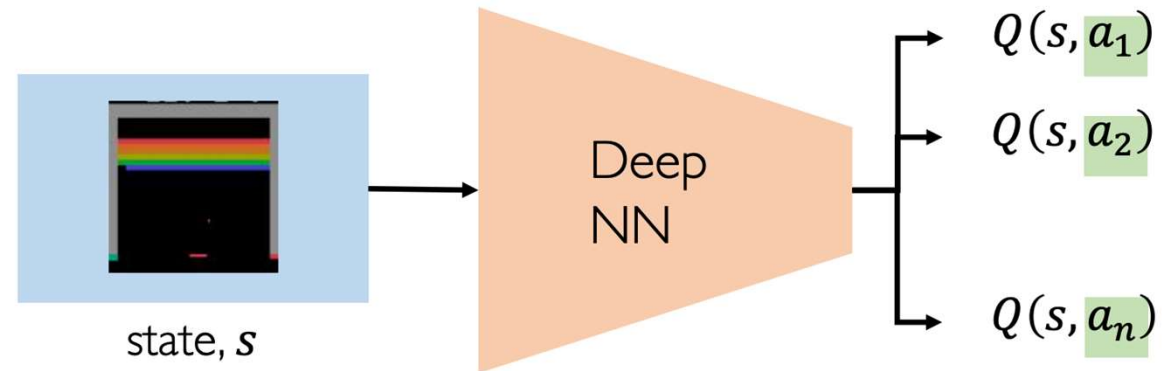
Q-function

$$R_t = \sum_{i=t}^{\infty} \gamma^i r_i = \gamma^t r_t + \gamma^{t+1} r_{t+1} \dots + \gamma^{t+n} r_{t+n} + \dots$$

$$Q(\overset{\uparrow}{\underset{\text{(state)}}{s}}, \overset{\uparrow}{\underset{\text{(action)}}{a}}) = \mathbb{E}[R_t]$$

$$\pi^*(\overset{\uparrow}{\underset{\text{(state)}}{s}}) = \underset{\underset{\text{(action)}}{a}}{\operatorname{argmax}} Q(\overset{\uparrow}{\underset{\text{(state)}}{s}}, \overset{\uparrow}{\underset{\text{(action)}}{a}})$$

Policy Gradient

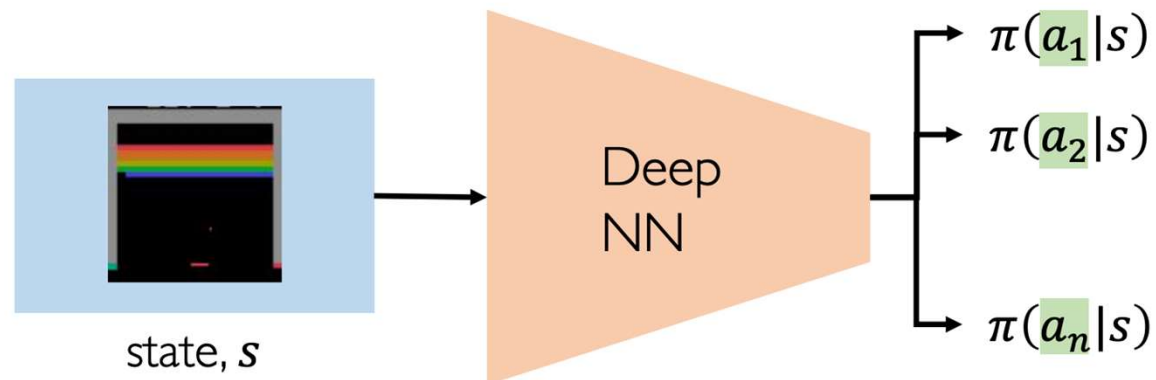


Policy Gradient

log-likelihood of action

$$\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R_t$$

reward



$$\sum_{a_i \in A} \pi(a_i | s) = 1$$

$$\pi(a|s) = P(\text{action}|\text{state})$$

Deep Reinforcement Learning Algorithms

Value Learning

Find $Q(s, a)$

$$a = \underset{a}{\operatorname{argmax}} Q(s, a)$$

Policy Learning

Find $\pi(s)$

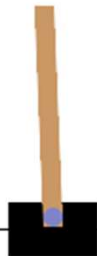
Sample $a \sim \pi(s)$

Action Space

The action is a `ndarray` with shape `(1,)` which can take values `{0, 1}` indicating the direction of the fixed force the cart is pushed with.

- 0: Push cart to the left
- 1: Push cart to the right

Note: The velocity that is reduced or increased by the applied force is not fixed and it depends on the angle the pole is pointing. The center of gravity of the pole varies the amount of energy needed to move the cart underneath it

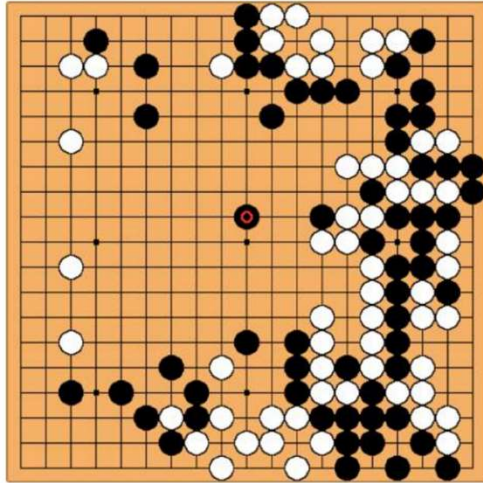


Observation Space

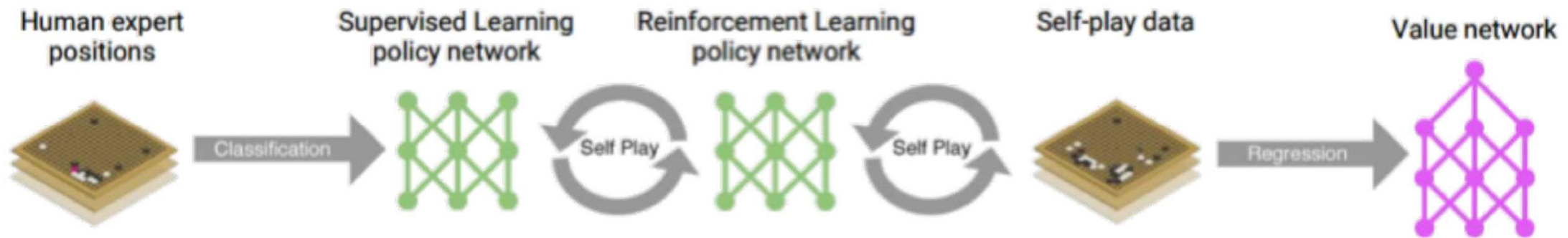
The observation is a `ndarray` with shape `(4,)` with the values corresponding to the following positions and velocities:

Num	Observation	Min	Max
0	Cart Position	-4.8	4.8
1	Cart Velocity	-Inf	Inf
2	Pole Angle	~ -0.418 rad (-24°)	~ 0.418 rad (24°)
3	Pole Angular Velocity	-Inf	Inf

Applications



Board Size $n \times n$	Positions 3^{n^2}	% Legal	Legal Positions
1×1	3	33.33%	1
2×2	81	70.37%	57
3×3	19,683	64.40%	12,675
4×4	43,046,721	56.49%	24,318,165
5×5	847,288,609,443	48.90%	414,295,148,741
9×9	$4.434264882 \times 10^{38}$	23.44%	$1.03919148791 \times 10^{38}$
13×13	$4.300233593 \times 10^{80}$	8.66%	$3.72497923077 \times 10^{79}$
19×19	$1.740896506 \times 10^{172}$	1.20%	$2.08168199382 \times 10^{170}$



小结

- **基于模型的强化学习**智能体需要（或者配备有）环境的转移模型 $P(s^t | s, a)$ ，并学习效用函数 $U(s)$.
- **无模型强化学习**智能体可以学习一个动作效用函数 $Q(s, a)$ 或学习一个策略 $\pi(s)$.
- 效用函数可以通过如下几种方法进行学习：
 - **直接效用估计**将观测到的总奖励用于给定状态，作为学习其效用的样本直接来源
 - **自适应动态规划 (ADP)** 从观测中学习模型和奖励函数，然后使用价值或策略迭代来获得效用或最优策略。ADP较好地利用了环境的邻接结构作为状态效用的局部约束
 - **时序差分 (TD) 方法**调整效用估计，使其与后继状态的效用估计相一致。它可以被视为ADP方法的一个简单近似，而且学习不需要预先知道转移模型。此外，使用一个学习模型来产生伪经验可以学习得更快
- **奖励设计**和**分层强化学习**有助于学习复杂的行为，特别是在奖励稀少且需要长动作序列才能获得奖励的情况下
- **策略搜索**方法直接对策略的表示进行操作，并试图根据观测到的表现对其进行改进。在随机领域中性能的剧烈变化是一个严重的问题，而在模拟领域中可以通过预先固定随机程度来克服这个难点。
- 当正确的奖励函数难以获得时，通过观测专家行为进行**学徒学习**是一种有效的解决方案。**模仿学习**将问题转换为从专家的状态-动作对中进行学习的监督学习问题。**逆强化学习**从专家的行为中推断有关奖励函数的信息。